Apostila

FinInsight Platform

Azure + Terraform + Databricks Lakehouse + AI/ML + GenAI

Passo a passo completo em português: do **terraform apply** ao modelo em produção, com código pronto para colar, caminho de cada arquivo e verificação em cada etapa.

Versão 3.0 - reestruturada em **sprints** (cada parte termina com algo rodando), com a **API nova de pipelines** (**pyspark.pipelines**), COPY INTO, Streaming Tables e Declarative Pipelines.

43 capítulos | 26 tabelas | 8 partes
Outubro de 2026

Sumário

Parte I - Fundamentos e Como Usar Esta Apostila 4

1. Como usar esta apostila e o que há de novo na v3.0 4
2. Objetivo, arquitetura e stack 6
3. Lakehouse, Delta Lake e Medallion 7
4. Preparando o ambiente (VS Code, Azure CLI, Terraform) 9
5. Estrutura de diretórios do repositório 10

Parte II - Sprint 1: Infraestrutura mínima funcionando 11

6. Login no Azure e backend remoto de state 11
7. Arquivos raiz do Terraform e o módulo storage 13
8. Módulo databricks (workspace + Access Connector) 16
9. Módulo unity-catalog (catálogo, schemas, landing e volume) 18
10. Executando init, validate, plan e apply 21
11. Confirmação no Portal Azure 22
12. Ambientes: tfvars versus databricks.yml 23

Parte III - Sprint 2: Primeiros dados na Bronze 24

13. Workspace, Git folder e credenciais do GitHub 24
14. Secrets, compute e chaves de API 25
15. Ingestão via API REST (notebook 01) 27
16. Auto Loader e pipelines declarativos (API nova) 31
17. Pipeline Bronze: do primeiro arquivo aos oito 33

Parte IV - Sprint 3: Formas de ingestão e streaming 37

18. Schema evolution e schema hints 37
19. COPY INTO 38
20. Streaming Tables standalone (SCHEDULE e REFRESH) 39
21. Declarative Pipelines (Lakeflow Spark Declarative Pipelines) 41
22. Comparativo: COPY INTO x Auto Loader x Streaming Table x Pipeline 43
23. Streaming com Event Hubs (Kafka) 44

Parte V - Sprint 4: Camada Silver 48

24. Delta Lake na prática 48
25. Silver: padronização e deduplicação 49
26. Silver: cálculos financeiros (retorno, SMA, RSI, z-score) 52
27. Silver: join multi-fonte e o pipeline Silver 54
28. CDC e SCD Type 2 55
29. Expectations e qualidade de dados 57

Parte VI - Sprint 5: Camada Gold 58

30. Catálogo de tabelas e star schema 58
31. Gold: dimensões 60
32. Gold: fatos de preço, retorno e volatilidade 62
33. Gold: risco (VaR, Sharpe, drawdown), sentimento e o pipeline Gold 64

Parte VII - Sprint 6: AI/ML e GenAI 67

34. Feature Engineering 67
35. Modelos preditivos com MLflow 68
36. Promoção de modelo e batch scoring 71
37. GenAI: insights, RAG e AI Functions 73

Parte VIII - Operação, Governança e Fechamento 76

38. Orquestração: Jobs e Databricks Asset Bundle 76
39. CI/CD DEV, HML e PROD 78
40. Governança, linhagem e mascaramento (LGPD) 80
41. Módulos opcionais: Key Vault e Azure OpenAI 81
42. Troubleshooting: todos os erros reais e as correções 82
43. Checklist, custos e glossário 84

Parte I - Fundamentos e Como Usar Esta Apostila

1. Como usar esta apostila e o que há de novo na v3.0

Esta é a **versão 3.0**. Ela foi reestruturada em **sprints**: cada parte termina com algo **rodando de verdade** e uma caixa de **ponto de verificação** dizendo o que você deve ter no fim. Você não precisa ler a apostila inteira antes de ver dados chegando.

> **NOVO NESTA VERSAO** Duas mudanças grandes na v3.0: (1) **ordem incremental** (infra mínima, depois a primeira tabela Bronze, depois as outras formas de ingestão, Silver, Gold, ML e GenAI); (2) todo o código de pipeline usa a **API nova** `from pyspark import pipelines as dp` (antigo `import dlt`).

O mapa das sprints

Sprint	Entrega	Capítulos
1 - Infra mínima	Storage, workspace Databricks e Unity Catalog com landing/volume no ar	6-12
2 - Primeiros dados	Coleta das APIs e pipeline Bronze com as 7 fontes de arquivo	13-17
3 - Ingestão	Schema evolution, COPY INTO, Streaming Table com SCHEDULE, pipelines SQL, comparativo e Event Hubs	18-23
4 - Silver	Delta na prática, cálculos financeiros, join multi-fonte, SCD2 e qualidade	24-29
5 - Gold	Star schema e KPIs de risco (VaR, Sharpe, drawdown)	30-33
6 - AI/ML e GenAI	Features, modelos, scoring, insights e RAG	34-37
Operação	Job + bundle, CI/CD, governança, opcionais, troubleshooting	38-43

Como reconhecer o que é novo

- **Barra laranja** em um bloco de código = código novo ou alterado. Barra preta = código estável.
- Caixas: **NOVO NESTA VERSAO**, **CORRIGIDO** (erro que apareceu na execução real), **CUSTO**, **CODIGO DE REFERENCIA** (não executado ainda).
- Toda caixa de código começa com uma barra que diz **o tipo, onde colar e o caminho do arquivo**.
- Os arquivos existem prontos no `fininsight-platform.zip`: o código do PDF é exatamente o do repositório.

Começando do zero: limpar o projeto antigo

Se você já tem uma versão anterior no Azure, **não apague a pasta local primeiro**: o state do Terraform fica no Azure e os recursos continuariam gerando custo. Faça nesta ordem:

1. No projeto **antigo**, dentro de `infra/`: `terraform destroy -var-file="envs/dev/terraform.tfvars"`. Isso remove workspace, storage, Event Hubs, Key Vault e Azure OpenAI.
2. No Portal, confirme que `rg-fininsight-dev` sumiu. O Databricks cria também um resource group gerenciado (`databricks-rg-...`); ele some junto com o workspace.
3. **Mantenha** o `rg-tfstate-fininsight` (storage do state): será reaproveitado. **Não apague o metastore** do Unity Catalog: ele é da conta, não do projeto.
4. Se o novo apply acusar nome em soft-delete (Key Vault ou Azure OpenAI), faça o purge: `az keyvault purge --name kv-fininsight-dev` ou `az cognitiveservices account purge ...`.
5. Só então apague a pasta local e o Git folder antigo, e comece no capítulo 2.

ATENCAO Se você apagar a pasta do Terraform **antes** do destroy, o state continua no Azure mas você perde o código; recrie-o do zip e rode o destroy mesmo assim.

O que foi executado de verdade e o que é referência

Parte	Estado
Terraform (storage, databricks, unity-catalog)	**Executado** com `apply` em eastus2. Landing/volume foram escritos depois, no mesmo padrão.
Notebooks 01 e 02, Bronze com `import dlt`	**Executados** no workspace; os erros encontrados estão na tabela do cap. 42. Nesta versão o código foi convertido para a API `dp` (mesma lógica).
Notebook 00 (cria o scope), módulo `eventhubs`, COPY INTO, Streaming Table, pipeline SQL	Escritos conforme a documentação; **ainda não executados**.
Silver e Gold	Escritos e revisados; fórmulas com teste de referência em pandas (`tests/`). **Execute o pipeline** para validar no Spark.
ML, GenAI, bundle e CI/CD	**Código de referência**: dependem de histórico de dados, permissões e endpoints do seu workspace.

Contagem de tabelas

Camada	Tabelas	Qtd.
Bronze	7 fontes de arquivo + `bz_cotacoes_streaming` (Event Hubs)	8
Silver	`sv_precos_ativos`, `sv_retornos_volatilidade`, `sv_precos_com_macro`, `sv_indicadores_macro`, `sv_cambio`, `sv_cripto`, `sv_noticias_processadas`, `sv_carteiras_scd2`	8
Gold	3 dimensões + 7 fatos	10
Total do modelo		**26**
Demonstração	`bz_b3_cotacoes_copyinto`, `bz_b3_cotacoes_sql`, `bz_b3_cotacoes_pipe`, `bz_composicao_carteiras_cdc`	4

2. Objetivo, arquitetura e stack

A **FinInsight Capital** é uma gestora fictícia que precisa consolidar dados de mercado (renda variável, câmbio, juros, cripto e notícias) em uma única plataforma analítica, saindo de planilhas dispersas para um Lakehouse no Azure Databricks.

- Ingerir dados públicos e gratuitos: BCB, B3, Yahoo Finance, Alpha Vantage, FRED, CoinGecko e NewsAPI;
- Calcular métricas financeiras reais na Silver/Gold: retorno, volatilidade, RSI, z-score, VaR, Sharpe e drawdown;
- Treinar modelos (classificação de risco, previsão do dólar, clusterização) com MLflow;
- Gerar insights em linguagem natural e sentimento de notícias com GenAI;
- Governar tudo com Unity Catalog e entregar por CI/CD (DEV, HML, PROD).

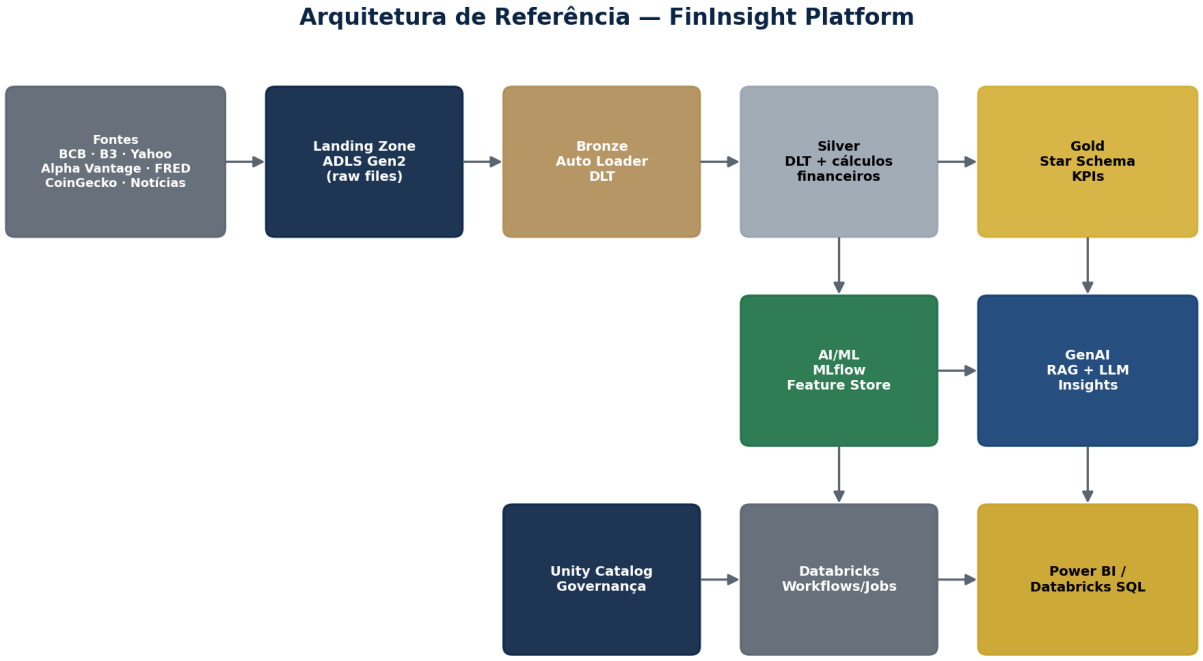


Figura 1 - Arquitetura de referência ponta a ponta.

Stack tecnológica

Camada	Tecnologia	Função
IaC	Terraform (azurerm + databricks)	Provisiona toda a infraestrutura Azure e o Unity Catalog
Storage	ADLS Gen2 (HNS)	Containers landing, bronze, silver, gold, unitycatalog
Ingestão	Auto Loader, COPY INTO, Streaming Tables, Event Hubs (Kafka)	Do arquivo bruto/stream até a Bronze
Processamento	Lakeflow Spark Declarative Pipelines (ex-DLT)	Bronze, Silver e Gold declarativas
Governança	Unity Catalog	Catálogo, linhagem, permissões, mascaramento
ML	MLflow + Feature Engineering (Unity Catalog)	Tracking, registry e scoring
GenAI	AI Functions, Vector Search, Azure OpenAI	Insights, RAG, sentimento
Orquestração e CI/CD	Lakeflow Jobs, Asset Bundles, GitHub/Bitbucket	Agendamento e deploy entre ambientes

> **ATENCAO** Nunca compartilhe catálogo ou Storage Account entre DEV e PROD. É uma das causas mais comuns de incidente.

3. Lakehouse, Delta Lake e Medallion

Lakehouse em uma página

Um **data lake** guarda arquivos baratos (Parquet, JSON, CSV), mas não garante consistência. Um **data warehouse** dá consistência e SQL rápido, mas custa mais e guarda pouco dado bruto. O **Lakehouse** junta os dois: arquivos abertos no storage da nuvem + uma camada transacional (**Delta Lake**) + governança (**Unity Catalog**) + motores de SQL, ETL e ML sobre os mesmos dados.

Delta Lake: o que ele adiciona ao Parquet

Conceito	O que significa	Por que importa aqui
Transaction log (`_delta_log`)	Histórico de operações em JSON versionado	Leituras concorrentes nunca veem dado parcial (ACID) enquanto o Auto Loader grava
Time travel	Consultar a tabela em uma versão/data passada	`SELECT ... VERSION AS OF 5`: investigar um KPI que mudou
Schema enforcement	A escrita falha se o schema não bate	Protege Silver/Gold de colunas corrompidas
Schema evolution	Adicionar colunas de forma controlada	Quando uma API muda o payload (cap. 18)
MERGE / upsert	Insere, atualiza e apaga em uma operação atômica	Base do CDC e do SCD2 (cap. 28)
OPTIMIZE + ZORDER	Compacta arquivos e reordena por colunas de filtro	Acelera consultas por `ticker` e data

Medallion: Bronze, Silver, Gold

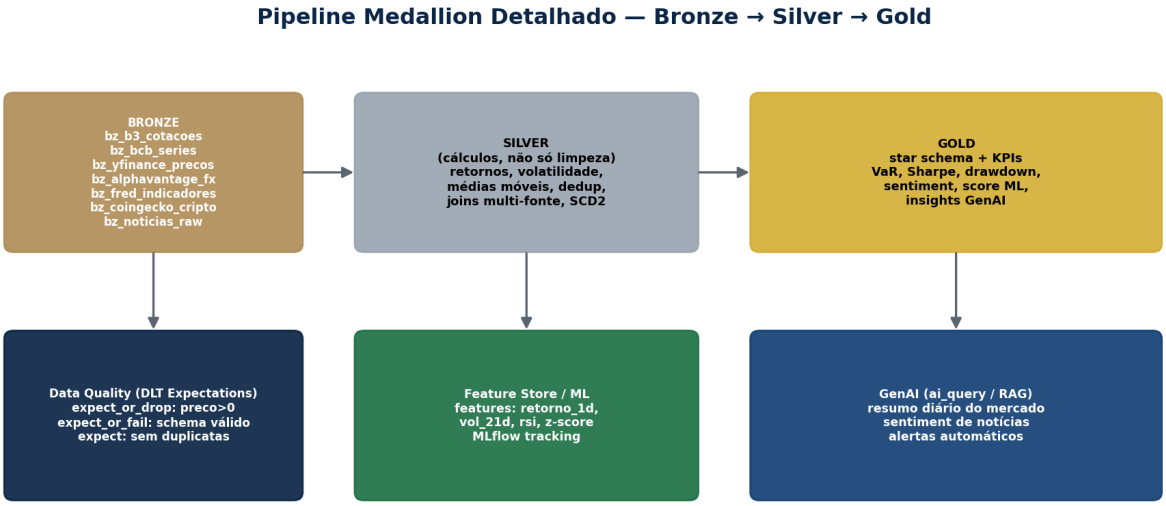


Figura 2 - Pipeline medallion detalhado.

Camada	Conteúdo	Regra de ouro
Bronze	Dado bruto, 1 tabela por fonte, com `_ingested_at` e `_source_file`	Não corrija nada: guarde como chegou
Silver	Padronizado, deduplicado e **com cálculos financeiros**	Aqui nascem retorno, SMA, RSI, z-score
Gold	Modelo dimensional (star schema) + KPIs de risco	Pronto para BI, ML e GenAI

Unity Catalog em uma página

- **Três níveis de nome**: `catalogo.schema.tabela`, por exemplo `fininsight_dev.bronze.bz_bcb_series`.
- **Managed table**: o Databricks controla arquivos e ciclo de vida. **External**: você aponta um caminho.
- **Volume**: pasta governada para arquivos não tabulares. O nosso `/Volumes/fininsight_dev/landing/raw/` é onde os JSON/CSV/Parquet brutos aterrissam.
- **Storage credential + external location**: o Access Connector (identidade gerenciada) ganha acesso ao container e o Unity Catalog passa a governar o caminho.

4. Preparando o ambiente (VS Code, Azure CLI, Terraform)

PASSO 1 **Instalar o VS Code e as extensões**

Baixe o VS Code (code.visualstudio.com). Em Extensions (Ctrl+Shift+X) instale: **HashiCorp Terraform**, **Azure Account** e, opcionalmente, **Databricks** (extensão oficial).

PASSO 2 **Instalar Azure CLI e Terraform**

TERMINAL | PowerShell/bash do seu sistema | instalação (uma vez)

```
# Windows (PowerShell como administrador)
winget install Microsoft.AzureCLI
winget install HashiCorp.Terraform

# macOS
brew install azure-cli terraform

# Linux (Ubuntu/Debian)
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
sudo apt-get install -y terraform
```

DICA Se o `terraform` não for encontrado depois de instalar, ele não está no PATH. No Windows você pode chamar o executável pelo caminho completo, por exemplo `& "C:\terraform\terraform.exe" version`.

PASSO 3 **Conferir as versões no terminal integrado**

TERMINAL | VS Code (Ctrl+` abre o terminal integrado) | dentro da pasta do projeto

```
az --version
terraform --version
```

ATENCAO Em **PowerShell** a continuação de linha é o acento grave (`), não a barra invertida (\). Comandos copiados de tutoriais de Linux com \` quebram. Neste material os comandos do Azure CLI vêm **em uma linha só**.

PASSO 4 **Extensão e CLI do Databricks (para o capítulo 38)**

TERMINAL | PowerShell | só será usado no cap. 38 (Asset Bundle)

```
winget install Databricks.DatabricksCLI
databricks --version
```

5. Estrutura de diretórios do repositório

Tudo nesta apostila assume esta árvore. Crie as pastas no VS Code (botão direito no Explorer, New Folder/New File) ou pelo terminal. Cada capítulo diz em qual arquivo colar o código.

ESTRUTURA | raiz do repositório | fininsight-platform/

```
fininsight-platform/
|-- README.md
|-- databricks.yml                    (cap. 38)  Asset Bundle: pipelines + job
|-- bitbucket-pipelines.yml           (cap. 39)
|-- .github/workflows/deploy.yml      (cap. 39)
|-- docs/diagramas/*.png
|-- infra/                            Terraform (Parte II)
|   |-- main.tf  backend.tf  variables.tf  outputs.tf
|   |-- envs/{dev,hml,prod}/terraform.tfvars
|   `-- modules/
|       |-- storage/  keyvault/  network/  databricks/
|       |-- unity-catalog/            (+ landing e volume)
|       |-- ai-services/
|       `-- eventhubs/                (novo, cap. 23)
|-- notebooks/
|   |-- 00_configurar_secrets.py      chaves de API (cap. 14)
|   |-- 01_coleta_apis.py             coleta das 7 fontes (cap. 15)
|   |-- 02_streaming_producer.py      producer Event Hubs (cap. 23)
|   |-- 04_feature_engineering.py     (cap. 34)
|   |-- 05_batch_scoring.py           (cap. 36)
|   `-- 06_genai_insights.py          (cap. 37)
|-- pipelines/
|   |-- bronze/      8 arquivos bz_*.py            (cap. 17)
|   |-- bronze_sql/  bz_b3_cotacoes_pipeline.sql   (cap. 21)
|   |-- silver/      8 arquivos sv_*.py            (cap. 25-28)
|   `-- gold/        dim_*.py e fato_*.py          (cap. 31-33)
|-- sql/             01..08 (COPY INTO, Streaming Table, Delta, governança...)
|-- ml/              classificador, previsão, clusterização, promoção
|-- genai/           vector_search_setup.py
`-- tests/           test_formulas_financeiras.py
```

PASSO 1 Criar a pasta do projeto e abrir no VS Code

TERMINAL | PowerShell ou bash | pasta onde você guarda seus projetos

```
mkdir fininsight-platform
cd fininsight-platform
code .
```

PASSO 2 Criar a árvore do Terraform

TERMINAL | PowerShell (Windows) | raiz do projeto

```
mkdir infra\modules\storage, infra\modules\keyvault, infra\modules\network
mkdir infra\modules\databricks, infra\modules\unity-catalog, infra\modules\ai-services
mkdir infra\modules\eventhubs
mkdir infra\envs\dev, infra\envs\hml, infra\envs\prod
mkdir notebooks, pipelines\bronze, pipelines\bronze_sql, pipelines\silver, pipelines\gold
mkdir sql, ml, genai, tests
```

TERMINAL | bash (macOS/Linux) | raiz do projeto

```
mkdir -p infra/modules/{storage,keyvault,network,databricks,unity-catalog,ai-services,eventhubs}
mkdir -p infra/envs/{dev,hml,prod}
mkdir -p notebooks pipelines/{bronze,bronze_sql,silver,gold}
mkdir -p sql ml genai tests
```

> **NOTA** Convenção de nomes: `bz_` para Bronze, `sv_` para Silver, `dim_`/`fato_` para Gold. Cada arquivo de pipeline define **uma tabela**; o pipeline lê a pasta inteira e monta o grafo de dependências sozinho.

Parte II - Sprint 1: Infraestrutura mínima funcionando

6. Login no Azure e backend remoto de state

PASSO 1 **Login no Azure**

TERMINAL | VS Code (terminal integrado) | qualquer pasta

```
az login
az account list --output table
az account set --subscription "NOME_OU_ID_DA_SUBSCRIPTION"
az account show --output table
```

O `az login` abre o navegador. Depois do `set`, o `show` confirma qual subscription está ativa: o Terraform usará essa.

PASSO 2 **Criar o storage do state (uma única vez, manualmente)**

O arquivo de state do Terraform nunca fica só no seu computador: ele vai para um container no Azure. Esse storage precisa existir **antes** do primeiro `terraform init`, por isso é criado à mão. A região escolhida foi **eastus2**, por ser a opção mais barata entre as avaliadas.

TERMINAL | PowerShell ou bash | uma linha por comando (sem barra invertida)

```
az group create -n rg-tfstate-fininsight -l eastus2
az storage account create -n tfstatefininsight001 -g rg-tfstate-fininsight -l eastus2 --sku Standard_LRS
    --encryption-services blob
az storage container create -n tfstate --account-name tfstatefininsight001 --auth-mode login
```

> **NOTA** `az group create -n ... -l ...` cria o Resource Group (pasta lógica que agrupa recursos) no local indicado. O nome do Storage Account precisa ser único no Azure, em minúsculas e sem hífen: se `tfstatefininsight001` já existir, troque o número e ajuste o `backend.tf`.

PASSO 3 **Criar infra/backend.tf**

Este arquivo diz ao Terraform onde guardar o state e quais providers usar. O provider `databricks` autentica pelo `azure_workspace_resource_id` do workspace que o próprio Terraform cria, então **não há token** para gerenciar.

TERRAFORM | VS Code, arquivo novo | infra/backend.tf

```
terraform {
  required_version = ">= 1.7.0"
  backend "azurerm" {
    resource_group_name  = "rg-tfstate-fininsight"
    storage_account_name = "tfstatefininsight001"
    container_name       = "tfstate"
    key                  = "fininsight.tfstate"
  }
  required_providers {
    azurerm    = { source = "hashicorp/azurerm", version = "~> 3.100" }
    databricks = { source = "databricks/databricks", version = "~> 1.50" }
  }
}

provider "azurerm" {
  features {}
}

provider "databricks" {
  host                        = module.databricks.workspace_url
  azure_workspace_resource_id = module.databricks.workspace_resource_id
}
```

CORRIGIDO A versão 1 tinha `key = "fininsight.${var.environment}.tfstate"`. **Variáveis não são permitidas no bloco backend.** O correto é uma `key` fixa; para outro ambiente, passe a chave no init: `terraform init -reconfigure -backend-config="key=fininsight.hml.tfstate"`.

ATENCAO Se um apply travar e o próximo falhar com `Error acquiring the state lock`, rode `terraform force-unlock <LOCK_ID>` (o ID aparece na mensagem de erro). Use somente se tiver certeza de que não há outro apply em andamento.

7. Arquivos raiz do Terraform e o módulo storage

Quatro arquivos na pasta `infra/` ligam os módulos. As variáveis mudam por ambiente (tfvars); o código é o mesmo. Esta sprint entrega o essencial: **storage, workspace Databricks e Unity Catalog com a landing**. Key Vault, Azure OpenAI e Event Hubs ficam para depois.

PASSO 1 infra/variables.tf

TERRAFORM | VS Code, arquivo novo | infra/variables.tf

```
variable "environment" {
  description = "Ambiente: dev, hml ou prod"
  type        = string
}

variable "location" {
  description = "Regiao Azure"
  type        = string
  default     = "eastus2"
}

variable "metastore_id" {
  description = "ID do metastore Unity Catalog ja existente na conta/regiao (copie do Account Console)"
  type        = string
}
```

PASSO 2 infra/main.tf

Cada bloco `module` chama uma pasta de `modules/` e liga as saídas de um módulo às entradas de outro (`module.storage.resource_group_name`, por exemplo). Nesta sprint ficam **ativos só três módulos**: storage, databricks e unity_catalog. Os módulos `eventhubs` (cap. 23), `keyvault` e `ai_services` (cap. 41) já estão no arquivo, **comentados**: o arquivo é completo desde já, e mais tarde você apenas remove o `#` do bloco que quiser ativar.

TERRAFORM | VS Code, arquivo novo | infra/main.tf

```
# ---------- Sprint 1: infraestrutura minima (ativa) ----------
module "storage" {
  source      = "./modules/storage"
  environment = var.environment
  location    = var.location
}

module "databricks" {
  source              = "./modules/databricks"
  environment         = var.environment
  location            = var.location
  resource_group_name = module.storage.resource_group_name
  resource_group_id   = module.storage.resource_group_id
  storage_account_id  = module.storage.storage_account_id
}

module "unity_catalog" {
  source               = "./modules/unity-catalog"
  environment          = var.environment
  location             = var.location
  storage_account_name = module.storage.storage_account_name
  access_connector_id  = module.databricks.access_connector_id
  metastore_id         = var.metastore_id

  depends_on = [module.storage, module.databricks]
}

# ---------- Sprint 3 (capitulo de streaming): descomente para ativar o Event Hubs ----------
# module "eventhubs" {
#   source              = "./modules/eventhubs"
#   environment         = var.environment
#   location            = var.location
#   resource_group_name = module.storage.resource_group_name
#
#   depends_on = [module.databricks]
# }

# ---------- Opcionais (capitulo "Modulos opcionais"): descomente quando quiser ----------
# module "keyvault" {
```

TERRAFORM | VS Code, arquivo novo | infra/main.tf

```
#   source              = "./modules/keyvault"
#   environment         = var.environment
#   location            = var.location
#   resource_group_name = module.storage.resource_group_name
# }
#
# module "ai_services" {
#   source              = "./modules/ai-services"
#   environment         = var.environment
#   location            = var.location
#   resource_group_name = module.storage.resource_group_name
# }
```

PASSO 3 infra/outputs.tf

TERRAFORM | VS Code, arquivo novo | infra/outputs.tf

```
output "resource_group_name" {
  value = module.storage.resource_group_name
}

output "databricks_workspace_url" {
  value = module.databricks.workspace_url
}

output "unity_catalog_name" {
  value = "fininsight_${var.environment}"
}
```

PASSO 4 infra/envs/dev/terraform.tfvars

Crie também `hml` e `prod` trocando `environment`. O `metastore_id` é o ID do metastore **já existente** na região (Account Console, Catalog, metastore, aba Configuration). O Databricks permite **um metastore por região por conta**.

ARQUIVO | VS Code, arquivo novo | infra/envs/dev/terraform.tfvars

```
environment  = "dev"
location     = "eastus2"
metastore_id = "COLE_AQUI_O_METASTORE_ID_DO_ACCOUNT_CONSOLE"
```

Módulo storage (ADLS Gen2)

Cria o Resource Group, o Storage Account com **namespace hierárquico** (ADLS Gen2) e cinco containers: `landing`, `bronze`, `silver`, `gold` e `unitycatalog` (raiz de armazenamento do catálogo).

TERRAFORM | VS Code, arquivo novo | infra/modules/storage/main.tf

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-fininsight-${var.environment}"
  location = var.location
}

resource "azurerm_storage_account" "adls" {
  name                     = "safininsight${var.environment}"
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  is_hns_enabled           = true
}

resource "azurerm_storage_container" "landing" {
  name                  = "landing"
  storage_account_name  = azurerm_storage_account.adls.name
  container_access_type = "private"
}

resource "azurerm_storage_container" "bronze" {
  name                  = "bronze"
  storage_account_name  = azurerm_storage_account.adls.name
```

TERRAFORM | VS Code, arquivo novo | infra/modules/storage/main.tf

```
  container_access_type = "private"
}

resource "azurerm_storage_container" "silver" {
  name                  = "silver"
  storage_account_name  = azurerm_storage_account.adls.name
  container_access_type = "private"
}

resource "azurerm_storage_container" "gold" {
  name                  = "gold"
  storage_account_name  = azurerm_storage_account.adls.name
  container_access_type = "private"
}

resource "azurerm_storage_container" "unitycatalog" {
  name                  = "unitycatalog"
  storage_account_name  = azurerm_storage_account.adls.name
  container_access_type = "private"
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/storage/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/storage/outputs.tf

```
output "resource_group_name" {
  value = azurerm_resource_group.rg.name
}

output "resource_group_id" {
  value = azurerm_resource_group.rg.id
}

output "storage_account_id" {
  value = azurerm_storage_account.adls.id
}

output "storage_account_name" {
  value = azurerm_storage_account.adls.name
}
```

CORRIGIDO O container `unitycatalog` não existia na v1. Sem ele, o catálogo com `storage_root` falhava na criação.

8. Módulo databricks (workspace + Access Connector)

Este módulo cria o **workspace** (SKU `premium`, exigido pelo Unity Catalog) e o **Access Connector**, a identidade gerenciada que o Databricks usa para acessar o storage. Quatro permissões são atribuídas a essa identidade:

Papel (role)	Escopo	Para quê
Storage Blob Data Contributor	Storage Account	Ler/gravar arquivos
Storage Queue Data Contributor	Storage Account	Fila do modo file notification do Auto Loader
EventGrid EventSubscription Contributor	**Resource Group**	Criar assinaturas de eventos de arquivo
Storage Account Contributor	**Resource Group**	Configurar notificações do storage

TERRAFORM | VS Code, arquivo novo | infra/modules/databricks/main.tf

```
resource "azurerm_databricks_workspace" "dbw" {
  name                = "dbw-fininsight-${var.environment}"
  resource_group_name = var.resource_group_name
  location            = var.location
  sku                 = "premium"
}

resource "azurerm_databricks_access_connector" "connector" {
  name                = "dbac-fininsight-${var.environment}"
  resource_group_name = var.resource_group_name
  location            = var.location
  identity { type = "SystemAssigned" }
}

resource "azurerm_role_assignment" "blob_contrib" {
  scope                = var.storage_account_id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_databricks_access_connector.connector.identity[0].principal_id
}

resource "azurerm_role_assignment" "queue_contrib" {
  scope                = var.storage_account_id
  role_definition_name = "Storage Queue Data Contributor"
  principal_id         = azurerm_databricks_access_connector.connector.identity[0].principal_id
}

resource "azurerm_role_assignment" "eventgrid_contrib" {
  scope                = var.resource_group_id
  role_definition_name = "EventGrid EventSubscription Contributor"
  principal_id         = azurerm_databricks_access_connector.connector.identity[0].principal_id
}

resource "azurerm_role_assignment" "storage_acct_contrib" {
  scope                = var.resource_group_id
  role_definition_name = "Storage Account Contributor"
  principal_id         = azurerm_databricks_access_connector.connector.identity[0].principal_id
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/databricks/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}

variable "resource_group_id" {
  type = string
}

variable "storage_account_id" {
  type = string
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/databricks/outputs.tf

```
output "workspace_id" {
  value = azurerm_databricks_workspace.dbw.workspace_id
}

output "workspace_url" {
  value = "https://${azurerm_databricks_workspace.dbw.workspace_url}"
}

output "access_connector_id" {
  value = azurerm_databricks_access_connector.connector.id
}

output "workspace_resource_id" {
  value = azurerm_databricks_workspace.dbw.id
}
```

CORRIGIDO SKU `standard` do Databricks foi descontinuado: usamos `premium`. As duas últimas roles precisam estar no escopo do **Resource Group**; no escopo do Storage Account o Azure responde `LinkedAuthorizationFailed`.

9. Módulo unity-catalog (catálogo, schemas, landing e volume)

O módulo conecta o storage ao Unity Catalog e cria tudo que o resto da apostila usa: **storage credential**, **external locations** (bronze, raiz do catálogo e landing), o **catálogo** `fininsight_<ambiente>`, os schemas `bronze`, `silver`, `gold`, `landing` e o **volume externo** `raw`.

> **NOVO NESTA VERSAO** Landing (external location + schema + volume) agora é criada pelo Terraform. Na v1 o schema `landing` não existia e o primeiro notebook falhava com `SCHEMA_NOT_FOUND fininsight_dev.landing`.

PASSO 1 infra/modules/unity-catalog/versions.tf

Todo módulo que usa um provider que **não** é da HashiCorp precisa declará-lo. Sem este arquivo o Terraform tenta baixar `hashicorp/databricks` e falha com `provider not found`.

TERRAFORM | VS Code, arquivo novo | infra/modules/unity-catalog/versions.tf

```
terraform {
  required_providers {
    databricks = {
      source  = "databricks/databricks"
      version = "~> 1.50"
    }
  }
}
```

PASSO 2 infra/modules/unity-catalog/main.tf

TERRAFORM | VS Code, arquivo novo (substitui a versão da v1) | infra/modules/unity-catalog/main.tf

```
# Credencial de storage: usa a identidade gerenciada do Access Connector
resource "databricks_storage_credential" "external" {
  name = "cred-fininsight-${var.environment}"
  azure_managed_identity {
    access_connector_id = var.access_connector_id
  }
}

# External locations (container -> credencial)
resource "databricks_external_location" "bronze" {
  name            = "loc-bronze-${var.environment}"
  url             = "abfss://bronze@${var.storage_account_name}.dfs.core.windows.net/"
  credential_name = databricks_storage_credential.external.name
}

resource "databricks_external_location" "catalog_root" {
  name            = "loc-catalog-root-${var.environment}"
  url             = "abfss://unitycatalog@${var.storage_account_name}.dfs.core.windows.net/"
  credential_name = databricks_storage_credential.external.name
}

# NOVO: container landing, onde os notebooks de coleta gravam os arquivos brutos
resource "databricks_external_location" "landing" {
  name            = "loc-landing-${var.environment}"
  url             = "abfss://landing@${var.storage_account_name}.dfs.core.windows.net/"
  credential_name = databricks_storage_credential.external.name
}

# Catalogo e schemas
resource "databricks_catalog" "fininsight" {
  name         = "fininsight_${var.environment}"
  comment      = "Catalogo da plataforma FinInsight"
  storage_root = databricks_external_location.catalog_root.url

  depends_on = [databricks_external_location.catalog_root]
}

resource "databricks_schema" "bronze" {
  catalog_name = databricks_catalog.fininsight.name
  name         = "bronze"
}
```

TERRAFORM | VS Code, arquivo novo (substitui a versão da v1) | infra/modules/unity-catalog/main.tf

```
resource "databricks_schema" "silver" {
  catalog_name = databricks_catalog.fininsight.name
  name         = "silver"
}

resource "databricks_schema" "gold" {
  catalog_name = databricks_catalog.fininsight.name
  name         = "gold"
}

# NOVO: schema landing + volume externo "raw" (caminho /Volumes/<catalogo>/landing/raw)
resource "databricks_schema" "landing" {
  catalog_name = databricks_catalog.fininsight.name
  name         = "landing"
}

resource "databricks_volume" "landing_raw" {
  catalog_name     = databricks_catalog.fininsight.name
  schema_name      = databricks_schema.landing.name
  name             = "raw"
  volume_type      = "EXTERNAL"
  storage_location = "abfss://landing@${var.storage_account_name}.dfs.core.windows.net/raw"
  comment          = "Arquivos brutos coletados das APIs (Auto Loader / COPY INTO leem daqui)"

  depends_on = [databricks_external_location.landing]
}
```

PASSO 3 infra/modules/unity-catalog/variables.tf

TERRAFORM | VS Code, arquivo novo | infra/modules/unity-catalog/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}

variable "storage_account_name" {
  type = string
}

variable "access_connector_id" {
  type = string
}

variable "metastore_id" {
  description = "ID do metastore existente na regiao (1 por regiao/conta). Associacao: Account Console."
  type        = string
}
```

CORRIGIDO Foram removidos `databricks_metastore` e `databricks_metastore_assignment`. Criar um segundo metastore na região falha (limite de um por região) e associar o workspace exige ser admin de conta/metastore. Se o seu workspace ainda não estiver associado, peça a um account admin: Account Console, Catalog, seu metastore, aba Workspaces, Assign to workspace.

NOTA Se você já criou o schema `landing` e o volume manualmente por SQL, não aplique este trecho (ou importe os recursos com `terraform import`), senão o Terraform acusa que eles já existem.

Para conferir o volume depois do apply, em um notebook ou no SQL Editor:

SQL | Databricks SQL Editor | verificação

```
SHOW VOLUMES IN fininsight_dev.landing;
-- esperado: raw   (EXTERNAL)
-- caminho de uso nos notebooks: /Volumes/fininsight_dev/landing/raw/
```

Azure OpenAI

Provisiona uma conta Azure OpenAI e o deployment do modelo **gpt-5-mini** (versão 2025-08-07). É opcional: as AI Functions do Databricks (cap. 37) funcionam sem este módulo.

TERRAFORM | VS Code, arquivo novo | infra/modules/ai-services/main.tf

```
resource "azurerm_cognitive_account" "openai" {
  name                = "aoai-fininsight-${var.environment}"
  resource_group_name = var.resource_group_name
  location            = var.location
  kind                = "OpenAI"
  sku_name            = "S0"
}

resource "azurerm_cognitive_deployment" "gpt4o_mini" {
  name                 = "gpt-5-mini-fininsight"
  cognitive_account_id = azurerm_cognitive_account.openai.id
  model {
    format  = "OpenAI"
    name    = "gpt-5-mini"
    version = "2025-08-07"
  }
  scale {
    type     = "GlobalStandard"
    capacity = 10
  }
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/ai-services/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}
```

CORRIGIDO Três erros reais aqui: (1) o bloco `sku` foi substituído por `scale`; (2) o `gpt-4o-mini` estava descontinuado, usamos **gpt-5-mini 2025-08-07**; (3) o tipo `Standard` não é aceito para esse modelo: use `GlobalStandard`. Disponibilidade de modelo varia por região e por assinatura; se o apply reclamar de quota, reduza `capacity` ou escolha outro modelo.

10. Executando init, validate, plan e apply

PASSO 1 **init, validate e plan**

TERMINAL | VS Code (terminal integrado) | dentro de infra/

```
cd infra
terraform init
terraform validate
terraform plan -var-file="envs/dev/terraform.tfvars" -out=plan.out
```

Leia o resumo do plano: `Plan: N to add, 0 to change, 0 to destroy`. Em um primeiro plano, não deve haver nada para alterar ou destruir.

ATENCAO Se `terraform plan` responder **"Too many command line arguments"** mesmo com a sintaxe correta, monte os argumentos em um array e chame o executável pelo caminho completo (era o caso do PowerShell com `C:\terraform\terraform.exe`):

TERMINAL | PowerShell | dentro de infra/ | alternativa

```
$argsList = @("plan", "-var-file=envs/dev/terraform.tfvars", "-out=plan.out")
& "C:\terraform\terraform.exe" $argsList
```

PASSO 2 **apply**

TERMINAL | VS Code (terminal integrado) | dentro de infra/

```
terraform apply plan.out
```

Leva de 5 a 15 minutos; o workspace Databricks é o recurso mais lento. Ao final, `Apply complete!` e os outputs (`databricks_workspace_url`, `unity_catalog_name`).

PASSO 3 **Como ficam os custos depois do apply**

- **Parado, quase de graça**: Resource Group, Storage (paga por GB), Access Connector, Key Vault, workspace sem compute.
- **Paga enquanto existir** (quando ativado): Event Hubs Standard (cap. 23), por hora de namespace.
- **Paga por uso**: compute dos pipelines/jobs, tokens do Azure OpenAI e das AI Functions.

PASSO 4 **Destruir (somente DEV, com plano revisado)**

TERMINAL | dentro de infra/ | nunca em PROD sem revisão

```
terraform plan -destroy -var-file="envs/dev/terraform.tfvars"
terraform destroy -var-file="envs/dev/terraform.tfvars"
# remover apenas o Event Hubs quando nao estiver usando:
terraform destroy -target=module.eventhubs -var-file="envs/dev/terraform.tfvars"
```

11. Confirmação no Portal Azure

Depois do apply, confira no Portal (portal.azure.com). Esta é a verificação que pega a causa mais comum de falha silenciosa: identidade e permissão.

1. **Resource Group** `rg-fininsight-dev` existe e contém: storage, workspace Databricks e Access Connector (Key Vault e Azure OpenAI só se você ativou os opcionais).
2. **Storage Account** `safininsightdev`, Containers: `landing`, `bronze`, `silver`, `gold`, `unitycatalog`.
3. **Access Connector** `dbac-fininsight-dev`, Identity: copie o Object ID.
4. **IAM da Storage Account**: roles *Storage Blob Data Contributor* e *Storage Queue Data Contributor* para esse Object ID.
5. **IAM do Resource Group**: *EventGrid EventSubscription Contributor* e *Storage Account Contributor*.
6. **Databricks, Catalog Explorer, External locations**: abra `loc-bronze-dev`, `loc-landing-dev`, `loc-catalog-root-dev` e clique **Test connection**: todos os itens verdes (Read, Write, List, Delete).
7. **Catalog Explorer**: catálogo `fininsight_dev` com schemas `bronze`, `silver`, `gold`, `landing`; em `landing` o volume `raw`.

Sintoma	Causa provável	Correção
Test connection falha em Write/Delete	Role atribuída no escopo errado ou ainda propagando	Esperar alguns minutos; reconferir IAM do Storage Account
External location não aparece	Metastore não associado ao workspace	Account Console: Assign to workspace
LinkedAuthorizationFailed no apply	EventGrid/Storage Account Contributor no escopo do Storage	Atribuir no **Resource Group**

12. Ambientes: tfvars versus databricks.yml

Duas coisas diferentes, que costumam ser confundidas, mudam entre DEV, HML e PROD:

	tfvars (Terraform)	databricks.yml (Asset Bundle)
O que gerencia	Infraestrutura: Azure + catálogo + schemas + volume	Código e jobs: pipelines, notebooks, agendamento
Quem aplica	`terraform apply -var-file=...`	`databricks bundle deploy -t dev|hml|prod`
O que muda por ambiente	`environment`, `location`, `metastore_id`	`host` do workspace, variável `catalog`, modo (`development`/`production`)
Frequência	Raramente (infra estável)	A cada merge (CI/CD)
Arquivo	`infra/envs/<amb>/terraform.tfvars`	`databricks.yml` (raiz), alvos `dev`, `hml`, `prod`

Fluxo completo de um ambiente novo: (1) `terraform apply` com o tfvars do ambiente; (2) copiar `databricks_workspace_url` para o `host` do alvo correspondente no `databricks.yml`; (3) `databricks bundle deploy -t <amb>`. O catálogo vira `fininsight_hml` ou `fininsight_prod` automaticamente (o Terraform cria; o bundle recebe pela variável `catalog`).

> **NOTA** Os notebooks e pipelines desta apostila usam `fininsight_dev` por clareza. Nos pipelines Silver/Gold o catálogo vem da configuração `fininsight.catalog` (cap. 27), então o mesmo código roda em HML/PROD só trocando a variável do bundle.

PONTO DE VERIFICACAO **Fim da Sprint 1: infraestrutura mínima no ar**

- `terraform apply` concluído sem erro e `terraform output` mostra a URL do workspace;
- Portal Azure: 5 containers, Access Connector e as 4 roles atribuídas;
- Catalog Explorer: catálogo `fininsight_dev` com schemas bronze, silver, gold, landing e o volume `landing.raw`;
- External locations com **Test connection** todo verde.

Parte III - Sprint 2: Primeiros dados na Bronze

13. Workspace, Git folder e credenciais do GitHub

PASSO 1 **Abrir o workspace**

Portal Azure, Resource Group, `dbw-fininsight-dev`, **Launch Workspace** (ou abra a URL do output `databricks_workspace_url`).

PASSO 2 **Criar a credencial do GitHub (Git folder)**

Databricks, canto superior direito, **Settings, Linked accounts** (ou User Settings, Git integration). Provider **GitHub**, método **Personal access token**. No GitHub gere um **token classic** em github.com/settings/tokens com o escopo `repo`, cole no Databricks e salve.

> **CORRIGIDO** Erro "The personal_access_token field cannot be set for GitHub Linked credentials": a credencial já estava vinculada ao app do GitHub. **Exclua a credencial** em Linked accounts e crie outra com Personal access token.

PASSO 3 **Clonar o repositório como Git folder**

Menu lateral, **Workspace**, seu usuário, **Create, Git folder**. Cole a URL HTTPS do repositório (por exemplo `https://github.com/<voce>/fininsight-platform.git`), provider GitHub, **Create Git folder**. Passe a trabalhar em uma **branch** (botão da branch no topo), nunca direto em `main`.

> **DICA** Fluxo de trabalho: edite no VS Code, faça `git push`, e no Databricks use **Pull** no Git folder. Também dá para editar dentro do Databricks e fazer commit/push por lá.

PASSO 4 **Conferir o catálogo no Catalog Explorer**

Menu **Catalog**: abra `fininsight_dev`; devem existir os schemas `bronze`, `silver`, `gold`, `landing` e o volume `landing.raw`.

Onde cada coisa roda

Código	Onde executa	Como
Terraform	Seu computador (VS Code)	Terminal integrado
Notebooks `notebooks/*.py`, `ml/*.py`	Databricks (serverless ou cluster)	Abrir o arquivo e Run all
Pipelines `pipelines/**`	Pipeline (compute próprio, serverless)	Jobs & Pipelines, Create, ETL pipeline
SQL `sql/*.sql`	SQL Editor (warehouse) ou notebook	Colar e executar
Bundle `databricks.yml`	Seu computador (Databricks CLI)	`databricks bundle deploy`

14. Secrets, compute e chaves de API

Chaves de API **nunca** vão no código. Ficam em um **secret scope** chamado `fininsight`, lido por `dbutils.secrets.get`.

> **NOTA** O próprio notebook 00 **cria** o scope `fininsight` na primeira execução (e não recria se já existir). Mais tarde, o módulo Terraform do Event Hubs (cap. 23) acrescenta nele os secrets de conexão.

PASSO 1 Obter as chaves (todas gratuitas)

Fonte	Onde obter	Secret (key)
Alpha Vantage	alphavantage.co/support/#api-key (o plano gratuito pode limitar alguns endpoints, inclusive `FX_DAILY`)	`alphavantage-api-key`
FRED	fred.stlouisfed.org/docs/api/api_key.html	`fred-api-key`
NewsAPI	newsapi.org/register (plano gratuito: sem uso comercial)	`newsapi-key`
CoinGecko	funciona sem chave; chave demo é opcional	`coingecko-key` (opcional)
BCB, B3, Yahoo Finance	públicas, sem chave	-

PASSO 2 Rodar o notebook 00 (uma vez)

Importe/abra `notebooks/00_configurar_secrets.py` no Databricks, cole as chaves no dicionário `CHAVES`, rode a célula e **apague as chaves do notebook** em seguida.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/00_configurar_secrets.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 00 - Configurar secrets (rodar UMA vez)
# MAGIC Cole as chaves abaixo, rode a célula e **apague as chaves do notebook depois**.
# MAGIC Este notebook cria o scope `fininsight` (se ainda não existir) e grava as chaves.
# MAGIC Os secrets do Event Hubs são adicionados depois pelo Terraform (módulo `eventhubs`).

# COMMAND ----------

from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
SCOPE = "fininsight"

# cria o scope uma unica vez (idempotente)
if SCOPE not in [s.name for s in w.secrets.list_scopes()]:
    w.secrets.create_scope(scope=SCOPE)

CHAVES = {
    "alphavantage-api-key": "COLE_AQUI",   # https://www.alphavantage.co/support/#api-key
    "fred-api-key": "COLE_AQUI",           # https://fred.stlouisfed.org/docs/api/api_key.html
    "newsapi-key": "COLE_AQUI",            # https://newsapi.org/register
}

for chave, valor in CHAVES.items():
    if valor != "COLE_AQUI":
        w.secrets.put_secret(scope=SCOPE, key=chave, string_value=valor)

print(sorted(s.key for s in w.secrets.list_secrets(SCOPE)))
```

PASSO 3 Escolher o compute

Opção	Quando usar	Observação
Serverless (notebook/job/pipeline)	Padrão deste projeto	Sem cluster para gerenciar; paga por uso
Cluster all-purpose pequeno (single node)	Desenvolvimento interativo prolongado	Auto-terminate em 30 min para não gastar parado

Opção	Quando usar	Observação
Job cluster	Jobs recorrentes sem serverless	Sobe e desce a cada execução

15. Ingestão via API REST (notebook 01)

Um notebook, `notebooks/01_coleta_apis.py`, coleta as 7 fontes e grava arquivos na landing, em pastas por fonte e por dia (`.../raw/<fonte>/dt=AAAA-MM-DD/`). Os arquivos são **append-only**: cada execução cria um arquivo novo.

Fonte	Formato gravado	Pasta	Observações
BCB (SGS 11, 433, 1)	NDJSON	`bcb/<serie>/`	A API aceita no máximo **20 registros** em `/ultimos/N`
Yahoo Finance	Parquet	`yfinance/`	Timestamp em **milissegundos** (`coerce_timestamps="ms"`)
B3 COTAHIST diário	CSV	`b3/`	Layout posicional; tenta os últimos 10 dias úteis
Alpha Vantage FX_DAILY	NDJSON	`alphavantage/`	Plano gratuito com limite baixo de chamadas/dia; `FX_DAILY` pode exigir plano pago
FRED	NDJSON	`fred/`	FEDFUNDS, CPIAUCSL, DGS10
CoinGecko	NDJSON	`coingecko/`	bitcoin, ethereum, solana
NewsAPI	NDJSON	`noticias/`	`noticia_id` = md5 da URL

PASSO 1 **Abrir e executar**

Abra o arquivo no Databricks (Git folder), conecte em compute serverless e use **Run all**. A saída esperada tem uma linha por fonte, `[OK]`, `[FALHA]` ou `[ignorado]` (sem chave). Fontes com falha fazem a última célula levantar erro listando quais.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/01_coleta_apis.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 01 - Coleta de APIs para a landing
# MAGIC Pré-requisitos: Volume `fininsight_dev.landing.raw` (Terraform) e secrets (notebook 00).

# COMMAND ----------

# MAGIC %pip install yfinance

# COMMAND ----------

dbutils.library.restartPython()

# COMMAND ----------

import hashlib
import io
import json
import os
import zipfile
from datetime import date, datetime, timedelta, timezone

import pandas as pd
import requests
import yfinance as yf

RAIZ = "/Volumes/fininsight_dev/landing/raw"
HOJE = date.today()
SCOPE = "fininsight"
# Primeira execucao (backfill): "2y". Execucoes diarias: "5d".
PERIODO_YF = "5d"


def segredo(chave):
    try:
        return dbutils.secrets.get(SCOPE, chave)
    except Exception:
        return None


def pasta_do_dia(fonte):
    pasta = f"{RAIZ}/{fonte}/dt={HOJE}"
    os.makedirs(pasta, exist_ok=True)
    return pasta
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/01_coleta_apis.py

```
def salvar_ndjson(fonte, registros, nome="dados.json"):
    # um objeto JSON por linha: formato que o Auto Loader lê sem configuração extra
    with open(f"{pasta_do_dia(fonte)}/{nome}", "w", encoding="utf-8") as f:
        for r in registros:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")
    return len(registros)

# COMMAND ----------

def coletar_bcb():
    series = {"selic": 11, "ipca": 433, "ptax_dolar": 1}
    total = 0
    for nome, codigo in series.items():
        # a API do BCB aceita no máximo 20 registros em /ultimos/N
        url = f"https://api.bcb.gov.br/dados/serie/bcdata.sgs.{codigo}/dados/ultimos/20?formato=json"
        r = requests.get(url, timeout=30)
        r.raise_for_status()
        registros = [{**d, "serie": nome} for d in r.json()]
        total += salvar_ndjson(f"bcb/{nome}", registros)
    return f"{total} registros"


def coletar_yfinance():
    tickers = ["PETR4.SA", "VALE3.SA", "ITUB4.SA", "^BVSP", "AAPL", "MSFT"]
    frames = []
    for t in tickers:
        d = yf.download(t, period=PERIODO_YF, interval="1d", auto_adjust=False, progress=False)
        if d.empty:
            continue
        if isinstance(d.columns, pd.MultiIndex):
            d.columns = d.columns.get_level_values(0)
        d = d.reset_index()
        d.columns.name = None
        d["ticker"] = t
        frames.append(d)
    if not frames:
        raise RuntimeError("yfinance nao retornou dados")
    df = pd.concat(frames, ignore_index=True)
    df.columns = [c.replace(" ", "_") for c in df.columns]   # "Adj Close" -> "Adj_Close"
    df["Date"] = pd.to_datetime(df["Date"]).dt.tz_localize(None)
    # o Spark nao le timestamp em nanossegundos: gravar em milissegundos
    df.to_parquet(
        f"{pasta_do_dia('yfinance')}/precos.parquet",
        coerce_timestamps="ms",
        allow_truncated_timestamps=True,
    )
    return f"{len(df)} linhas"


def coletar_b3():
    # arquivo diario COTAHIST da B3: tenta os ultimos dias uteis ate achar um publicado
    for i in range(10):
        dia = HOJE - timedelta(days=i)
        if dia.weekday() >= 5:
            continue
        url = f"https://bvmf.bmfbovespa.com.br/InstDados/SerHist/COTAHIST_D{dia:%d%m%Y}.ZIP"
        r = requests.get(url, timeout=120)
        if r.status_code != 200:
            continue
        with zipfile.ZipFile(io.BytesIO(r.content)) as z:
            texto = z.read(z.namelist()[0]).decode("latin-1")
        linhas = []
        for ln in texto.splitlines():
            # registro tipo 01 (cotacao) e mercados 010 (vista) / 020 (fracionario)
            if ln[0:2] != "01" or ln[24:27] not in ("010", "020"):
                continue
            linhas.append({
                "codigo": ln[12:24].strip(),
                "data_pregao": f"{ln[2:6]}-{ln[6:8]}-{ln[8:10]}",
                "preco_abertura": int(ln[56:69]) / 100,
                "preco_maximo": int(ln[69:82]) / 100,
                "preco_minimo": int(ln[82:95]) / 100,
                "preco_fechamento": int(ln[108:121]) / 100,
                "volume": int(ln[170:188]) / 100,
            })
        if not linhas:
            continue
        pd.DataFrame(linhas).to_csv(f"{pasta_do_dia('b3')}/cotacoes.csv", index=False)
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/01_coleta_apis.py

```
            return f"{len(linhas)} linhas ({dia})"
    raise RuntimeError("nenhum arquivo COTAHIST diario encontrado nos ultimos 10 dias")


def coletar_alphavantage():
    chave = segredo("alphavantage-api-key")
    if not chave:
        return "ignorado (sem chave alphavantage-api-key)"
    r = requests.get(
        "https://www.alphavantage.co/query",
        params={"function": "FX_DAILY", "from_symbol": "USD", "to_symbol": "BRL",
                "outputsize": "compact", "apikey": chave},
        timeout=30,
    )
    r.raise_for_status()
    corpo = r.json()
    serie = corpo.get("Time Series FX (Daily)")
    if not serie:
        raise RuntimeError(f"resposta inesperada do Alpha Vantage: {str(corpo)[:300]}")
    registros = [{
        "par": "USDBRL", "data": dia,
        "open": float(v["1. open"]), "high": float(v["2. high"]),
        "low": float(v["3. low"]), "close": float(v["4. close"]),
    } for dia, v in serie.items()]
    return f"{salvar_ndjson('alphavantage', registros)} registros"


def coletar_fred():
    chave = segredo("fred-api-key")
    if not chave:
        return "ignorado (sem chave fred-api-key)"
    registros = []
    for serie in ["FEDFUNDS", "CPIAUCSL", "DGS10"]:
        r = requests.get(
            "https://api.stlouisfed.org/fred/series/observations",
            params={"series_id": serie, "api_key": chave, "file_type": "json",
                    "sort_order": "desc", "limit": 24},
            timeout=30,
        )
        r.raise_for_status()
        registros += [{"serie": serie, "data": o["date"], "valor": o["value"]}
                      for o in r.json()["observations"]]
    return f"{salvar_ndjson('fred', registros)} registros"


def coletar_coingecko():
    headers = {}
    chave = segredo("coingecko-key")
    if chave:
        headers["x-cg-demo-api-key"] = chave
    r = requests.get(
        "https://api.coingecko.com/api/v3/coins/markets",
        params={"vs_currency": "usd", "ids": "bitcoin,ethereum,solana"},
        headers=headers, timeout=30,
    )
    r.raise_for_status()
    agora = datetime.now(timezone.utc).isoformat()
    registros = [{
        "id": c["id"], "simbolo": c["symbol"], "preco_usd": c["current_price"],
        "market_cap": c["market_cap"], "volume_24h": c["total_volume"], "coletado_em": agora,
    } for c in r.json()]
    return f"{salvar_ndjson('coingecko', registros)} registros"


def coletar_noticias():
    chave = segredo("newsapi-key")
    if not chave:
        return "ignorado (sem chave newsapi-key)"
    r = requests.get(
        "https://newsapi.org/v2/everything",
        params={"q": "Petrobras OR Vale OR Itau OR Ibovespa OR Selic", "language": "pt",
                "sortBy": "publishedAt", "pageSize": 50, "apiKey": chave},
        timeout=30,
    )
    r.raise_for_status()
    mapa = {"Petrobras": "PETR4.SA", "Vale": "VALE3.SA", "Ita": "ITUB4.SA"}
    registros = []
    for a in r.json().get("articles", []):
        texto = " ".join(filter(None, [a.get("title"), a.get("description"), a.get("content")]))
        registros.append({
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/01_coleta_apis.py

```
            "noticia_id": hashlib.md5(a["url"].encode()).hexdigest(),
            "titulo": a.get("title"),
            "texto_noticia": texto,
            "fonte": (a.get("source") or {}).get("name"),
            "data_publicacao": a.get("publishedAt"),
            "url": a["url"],
            "ticker_relacionado": next((t for k, t in mapa.items() if k in texto), None),
        })
    return f"{salvar_ndjson('noticias', registros)} registros"

# COMMAND ----------

tarefas = {
    "bcb": coletar_bcb,
    "yfinance": coletar_yfinance,
    "b3": coletar_b3,
    "alphavantage": coletar_alphavantage,
    "fred": coletar_fred,
    "coingecko": coletar_coingecko,
    "noticias": coletar_noticias,
}

falhas = {}
for nome, fn in tarefas.items():
    try:
        print(f"[OK]    {nome}: {fn()}")
    except Exception as e:
        falhas[nome] = repr(e)[:300]
        print(f"[FALHA] {nome}: {falhas[nome]}")

if falhas:
    raise RuntimeError(f"fontes com falha: {list(falhas)}")
```

CORRIGIDO Erros reais resolvidos neste notebook: `No module named yfinance` (`%pip install yfinance` + `restartPython`), `OSError: Cannot save file into a non-existent directory` (`os.makedirs`), BCB `HTTP 400` (máximo de 20 registros) e `Illegal Parquet type INT64 (TIMESTAMP(NANOS,false))` no pipeline (timestamps em ms).

ATENCAO Se você coletou o yfinance com a versão antiga, apague o parquet inválido antes de recoletar: `dbutils.fs.rm("/Volumes/fininsight_dev/landing/raw/yfinance/", recurse=True)`.

Backfill: histórico para as métricas financeiras

Com `period="5d"` o yfinance entrega só 5 dias: insuficiente para SMA de 63 dias ou volatilidade de 21 dias. Na **primeira execução**, mude a constante no topo do notebook para `PERIODO_YF = "2y"`, rode, e volte para `"5d"` no agendamento diário. O Auto Loader processa os novos arquivos e a Silver passa a ter janelas completas.

CODIGO DE REFERENCIA O endereço do COTAHIST diário da B3 (`bvmf.bmfbovespa.com.br/InstDados/SerHist/...`) e as posições do layout fixo foram escritos conforme o layout público, mas a URL pode mudar. Se a fonte B3 falhar, o notebook acusa e as demais continuam. Para mais histórico do BCB (não testado), a API aceita `dataInicial` e `dataFinal` no formato dd/MM/aaaa.

16. Auto Loader e pipelines declarativos (API nova)

Auto Loader

Auto Loader é a fonte de streaming `cloudFiles`. Ele descobre **somente arquivos novos** de uma pasta, lembra o que já processou (checkpoint gerenciado) e infere/evolui o schema. É a forma recomendada de ingerir arquivos para a Bronze.

Opção	Efeito
`cloudFiles.format`	Formato dos arquivos: `json`, `csv`, `parquet`...
`cloudFiles.inferColumnTypes`	Infere tipos (sem isso tudo vira string em JSON/CSV)
`cloudFiles.schemaEvolutionMode`	O que fazer quando aparece coluna nova (cap. 18)
`_metadata.file_path`	Coluna de metadados com o caminho do arquivo de origem (use no lugar de `input_file_name()`)

Pipelines declarativos (antigo Delta Live Tables)

Um **pipeline declarativo** (Lakeflow Spark Declarative Pipelines, o antigo DLT) executa uma **pasta de arquivos** e monta o grafo de dependências a partir do que cada tabela lê. Você declara **o quê**; o Databricks cuida de ordem, checkpoints, retries e infraestrutura.

Objeto	Como é criado (Python)	Comportamento
Streaming table	`@dp.table` retornando um **streaming DataFrame** (`readStream`)	Incremental: processa só o que é novo, uma vez
Materialized view	`@dp.materialized_view` retornando DataFrame **batch** (`spark.read`)	Resultado mantido e atualizado de forma incremental quando possível
View temporária	`@dp.temporary_view`	Não é gravada; só organiza a lógica
Expectation	`@dp.expect`, `@dp.expect_or_drop`, `@dp.expect_or_fail`	Regra de qualidade com métricas (cap. 29)

A API nova: `pyspark.pipelines`

Todo o código desta apostila usa a API atual: `from pyspark import pipelines as dp`. Para ler uma tabela do próprio pipeline basta `spark.read.table("nome")` (ou `spark.readStream.table`), sem `dlt.read` e sem prefixo `LIVE.`; o nome sem catálogo/schema resolve para o catálogo e schema padrão do pipeline.

API antiga (import dlt)	API nova (pyspark.pipelines as dp)
`@dlt.table` (streaming)	`@dp.table`
`@dlt.table` (batch, vira materialized view)	`@dp.materialized_view`
`@dlt.view`	`@dp.temporary_view`
`dlt.read("t")` / `dlt.read_stream("t")`	`spark.read.table("t")` / `spark.readStream.table("t")`
`dlt.create_streaming_table`	`dp.create_streaming_table`
`dlt.apply_changes`	`dp.create_auto_cdc_flow`
`@dlt.expect*`	`@dp.expect*`

> **NOTA** Se o seu runtime/pipeline for antigo e `from pyspark import pipelines` não existir, troque pelo equivalente da coluna da esquerda (`import dlt`): a lógica é a mesma. Pipelines **serverless** criados hoje já trazem a API nova.

O pipeline roda em compute separado?

Sim. Cada pipeline tem seu **próprio compute**, que sobe quando você clica Start (ou quando um agendamento dispara) e desce ao terminar. Em **serverless** você não escolhe máquinas; em classic você configura o cluster do pipeline. O notebook que você abre no editor **não executa** o pipeline: o que executa é o pipeline, apontando para a pasta.

> **CUSTO** Pipeline serverless cobra por uso enquanto roda. Em desenvolvimento, use o modo **Development** do pipeline (reutiliza compute e não faz retry automático) e rode sob demanda.

17. Pipeline Bronze: do primeiro arquivo aos oito

Os 7 arquivos de fonte têm exatamente o mesmo padrão; muda só o caminho e o formato. Vamos devagar: primeiro **um** arquivo, rodando, e depois os outros seis. O 8º (`bz_cotacoes_streaming.py`, Event Hubs) só entra no cap. 23.

> **CORRIGIDO** Dois erros da v1: (1) `cloudFiles.schemaLocation` apontando para um Volume inexistente (`Volume bronze._schemas does not exist`): **dentro do pipeline o schema é gerenciado**, não configure `schemaLocation`; (2) `input_file_name()` não é suportado no Unity Catalog: use `col("_metadata.file_path")`.

PASSO 1 Primeiro arquivo: bz_bcb_series.py

Crie `pipelines/bronze/bz_bcb_series.py` no VS Code (ou no Git folder). É um arquivo `.py` simples, **não** um notebook. Pré-requisito: você já rodou o notebook 01 do cap. 15 e há arquivos em `/Volumes/fininsight_dev/landing/raw/bcb/`.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_bcb_series.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_bcb_series",
    comment="Series temporais do Banco Central (raw)",
    table_properties={"quality": "bronze"},
)
def bz_bcb_series():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/bcb/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

PASSO 2 Criar o pipeline e rodar

1. Menu **Jobs & Pipelines**, **Create, ETL pipeline**.
2. Nome: `fininsight_bronze`. **Default catalog**: `fininsight_dev`. **Default schema**: `bronze`.
3. **Source code**: **Add existing assets** e aponte a **pasta** `pipelines/bronze` do Git folder (não um arquivo).
4. Se o assistente criou um arquivo de exemplo (`my_transformation.py` ou semelhante), **remova-o** da lista de source code.
5. Compute: **Serverless**. Modo: **Development** enquanto estiver testando.
6. Clique **Start**. O grafo mostra a tabela em verde ao terminar.

SQL | Databricks SQL Editor | verificação da primeira tabela (esperado: selic, ipca, ptax_dolar)

```
SELECT serie, count(*) AS linhas, min(data) AS primeira, max(data) AS ultima
FROM fininsight_dev.bronze.bz_bcb_series
GROUP BY serie;
```

PASSO 3 Os outros seis arquivos

Crie os arquivos abaixo em `pipelines/bronze/` e clique **Start** de novo. Três deles dependem de chave de API (Alpha Vantage, FRED, NewsAPI): se você ainda não configurou a chave no cap. 14, **deixe esse arquivo para depois**, porque a pasta dele na landing não terá dados e o pipeline falharia.

bz_b3_cotacoes (CSV do COTAHIST)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_b3_cotacoes.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_b3_cotacoes",
    comment="Cotacoes diarias da B3 - COTAHIST (raw)",
    table_properties={"quality": "bronze"},
)
def bz_b3_cotacoes():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/b3/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

bz_yfinance_precos (Parquet do Yahoo Finance)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_yfinance_precos.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_yfinance_precos",
    comment="Precos diarios via yfinance (raw)",
    table_properties={"quality": "bronze"},
)
def bz_yfinance_precos():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "parquet")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/yfinance/")
        .withColumnRenamed("Adj Close", "Adj_Close")  # Delta nao aceita espaco no nome da coluna
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

bz_alphavantage_fx (JSON do Alpha Vantage)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_alphavantage_fx.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_alphavantage_fx",
    comment="Cambio USD/BRL via Alpha Vantage (raw)",
    table_properties={"quality": "bronze"},
)
def bz_alphavantage_fx():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/alphavantage/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

bz_fred_indicadores (JSON do FRED)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_fred_indicadores.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_fred_indicadores",
    comment="Indicadores macro dos EUA - FRED (raw)",
    table_properties={"quality": "bronze"},
)
def bz_fred_indicadores():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/fred/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

bz_coingecko_cripto (JSON do CoinGecko)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_coingecko_cripto.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_coingecko_cripto",
    comment="Preco, volume e market cap de cripto - CoinGecko (raw)",
    table_properties={"quality": "bronze"},
)
def bz_coingecko_cripto():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/coingecko/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

bz_noticias_raw (JSON do NewsAPI)

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ | pipelines/bronze/bz_noticias_raw.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_noticias_raw",
    comment="Noticias financeiras brutas (raw)",
    table_properties={"quality": "bronze"},
)
def bz_noticias_raw():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .load("/Volumes/fininsight_dev/landing/raw/noticias/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

> **NOTA** `bz_yfinance_precos.py` renomeia `Adj Close` para `Adj_Close`: o Delta não aceita espaço em nome de coluna. É uma proteção preventiva (o coletor do cap. 15 já grava sem espaço).

PASSO 4 Verificar as 7 tabelas

SQL | Databricks SQL Editor | verificação das 7 tabelas de arquivo

```
SELECT 'b3' AS fonte, count(*) AS linhas FROM fininsight_dev.bronze.bz_b3_cotacoes
UNION ALL SELECT 'bcb', count(*) FROM fininsight_dev.bronze.bz_bcb_series
UNION ALL SELECT 'yfinance', count(*) FROM fininsight_dev.bronze.bz_yfinance_precos
UNION ALL SELECT 'alphavantage', count(*) FROM fininsight_dev.bronze.bz_alphavantage_fx
UNION ALL SELECT 'fred', count(*) FROM fininsight_dev.bronze.bz_fred_indicadores
UNION ALL SELECT 'coingecko', count(*) FROM fininsight_dev.bronze.bz_coingecko_cripto
UNION ALL SELECT 'noticias', count(*) FROM fininsight_dev.bronze.bz_noticias_raw;
```

Rode o pipeline uma segunda vez sem coletar nada: as contagens **não mudam**. É o Auto Loader provando que só processa arquivo novo.

PONTO DE VERIFICACAO Fim da Sprint 2: dados chegando na Bronze

- Git folder clonado e secrets configurados (`dbutils.secrets.list("fininsight")`);
- Notebook 01 com `[OK]` nas fontes que não precisam de chave (BCB, yfinance, B3, CoinGecko);
- Pipeline `fininsight_bronze` rodando e as tabelas `bz_*` com linhas;
- Segunda execução sem arquivo novo não altera as contagens.

Parte IV - Sprint 3: Formas de ingestão e streaming

18. Schema evolution e schema hints

Fontes mudam: uma API ganha um campo novo, um CSV ganha uma coluna. O Auto Loader tem quatro respostas, controladas por `cloudFiles.schemaEvolutionMode`:

Modo	Comportamento	Quando usar
`addNewColumns` (padrão com inferência)	O stream **falha uma vez** ao ver coluna nova, registra o novo schema e, ao reiniciar, segue com ela	Bronze: queremos receber tudo
`rescue`	Nunca falha; dados fora do schema vão para a coluna `_rescued_data` (JSON)	Schema fixo, mas sem perder nada
`failOnNewColumns`	Falha até você atualizar o schema manualmente	Contratos rígidos
`none`	Ignora colunas novas	Raramente

> **NOTA** No pipeline declarativo, a falha de `addNewColumns` é tratada pelo **retry automático**: a coluna nova aparece sem você intervir. Em um notebook comum de streaming, você precisaria reiniciar o stream manualmente (e usar `schemaLocation`).

Schema hints: fixar tipos que a inferência erra

A inferência pode ler `volume` como inteiro quando o dia seguinte traz decimal. `cloudFiles.schemaHints` fixa o tipo de colunas específicas e deixa o resto inferido. Arquivo completo (é a variante opcional de `bz_b3_cotacoes.py`):

PIPELINE DECLARATIVO (PYTHON) | VS Code, substitui pipelines/bronze/bz_b3_cotacoes.py (opcional) | pipelines/bronze/bz_b3_cotacoes.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, current_timestamp


@dp.table(
    name="bz_b3_cotacoes",
    comment="Cotacoes diarias da B3 - COTAHIST (raw), com tipos fixados",
    table_properties={"quality": "bronze"},
)
def bz_b3_cotacoes():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
        .option("cloudFiles.schemaHints",
                "preco_abertura DOUBLE, preco_maximo DOUBLE, preco_minimo DOUBLE, "
                "preco_fechamento DOUBLE, volume DOUBLE, data_pregao DATE")
        .load("/Volumes/fininsight_dev/landing/raw/b3/")
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", col("_metadata.file_path"))
    )
```

> **ATENCAO** Mudar o tipo de uma coluna **já existente** na tabela Bronze exige **Full refresh** do pipeline (Start, seta ao lado, Full refresh selected). Colunas novas não exigem.

19. COPY INTO

> **NOVO NESTA VERSAO** Capítulo novo. COPY INTO é o comando SQL de carga de arquivos para uma tabela Delta, **idempotente**: cada arquivo é carregado uma única vez.

Como funciona

- A tabela de destino é uma **tabela Delta comum** (você a cria; não é gerenciada por pipeline).
- O Databricks guarda quais arquivos já entraram. Rodar o comando de novo **ignora** os já carregados e traz só os novos.
- Sem estado de streaming: é um comando em lote, que você dispara quando quiser (SQL Editor, notebook, job).
- Bom para **centenas ou milhares** de arquivos; para volumes muito maiores, o Auto Loader escala melhor.

Opção	Efeito
`FILEFORMAT`	CSV, JSON, PARQUET, AVRO, TEXT, BINARYFILE
`FORMAT_OPTIONS`	Opções do leitor: `header`, `inferSchema`, `delimiter`...
`COPY_OPTIONS ('mergeSchema'='true')`	Evolui o schema da tabela se aparecer coluna nova
`COPY_OPTIONS ('force'='true')`	Recarrega arquivos já carregados (use com cuidado: duplica linhas)
`PATTERN` / `FILES`	Filtra por glob ou lista explícita de arquivos

PASSO 1 Executar no SQL Editor

Abra o **SQL Editor**, escolha um warehouse (serverless) e rode o script inteiro. A tabela é `bz_b3_cotacoes_copyinto`, uma cópia de **demonstração** da Bronze da B3: nenhuma tabela Silver depende dela.

SQL | Databricks SQL Editor, colar tudo e Run | sql/01_copy_into_b3.sql

```
-- Databricks SQL Editor (ou celula %sql de notebook). Cole tudo e execute.
-- 1) A tabela de destino precisa existir; sem colunas, o COPY INTO cria o schema na primeira carga.
CREATE TABLE IF NOT EXISTS fininsight_dev.bronze.bz_b3_cotacoes_copyinto;

-- 2) Carga incremental e idempotente: arquivos ja carregados sao ignorados.
COPY INTO fininsight_dev.bronze.bz_b3_cotacoes_copyinto
FROM '/Volumes/fininsight_dev/landing/raw/b3/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true', 'mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');

-- 3) Conferir o resultado (a tabela de demonstracao nao alimenta a Silver)
SELECT count(*) AS linhas FROM fininsight_dev.bronze.bz_b3_cotacoes_copyinto;
DESCRIBE HISTORY fininsight_dev.bronze.bz_b3_cotacoes_copyinto;

-- 4) Rode o COPY INTO de novo: num_affected_rows = 0, porque nenhum arquivo novo apareceu.
-- Para recarregar tudo de proposito, use COPY_OPTIONS ('force' = 'true').
```

PASSO 2 Provar a idempotência

1. Rode o COPY INTO e anote `num_affected_rows` (linhas carregadas).
2. Rode **de novo**: `num_affected_rows = 0`.
3. Execute o notebook 01 mais uma vez (gera um arquivo novo na landing) e rode o COPY INTO: entram **só** as linhas do arquivo novo.
4. `DESCRIBE HISTORY` mostra uma versão por carga.

> **CODIGO DE REFERENCIA** Texto da v2 conforme a documentação do Databricks; ainda não executado no seu workspace. Se `inferSchema` trouxer tipos diferentes dos do Auto Loader, é esperado: o COPY INTO usa seu próprio inferidor.

20. Streaming Tables standalone (SCHEDULE e REFRESH)

> **NOVO NESTA VERSAO** Capítulo novo. Uma **Streaming Table standalone** é criada direto em SQL, sem arquivo de pipeline, e pode ter **agendamento embutido** (`SCHEDULE`).

O que ela é

É a mesma ideia da tabela `@dp.table` de streaming (incremental, uma vez por arquivo), mas declarada em SQL com `CREATE OR REFRESH STREAMING TABLE`. Por trás, o Databricks cria **um pipeline serverless dedicado e invisível** para essa tabela; você a gerencia só com SQL.

Aspecto	Streaming Table standalone
Onde se cria	SQL Editor (warehouse) ou notebook SQL
Fonte típica	`STREAM read_files(...)` (Auto Loader por baixo), tabelas Delta, Kafka
Compute	Pipeline serverless dedicado (gerenciado), cobrado por uso
Agendamento	Cláusula `SCHEDULE` (cria um job automaticamente) ou `REFRESH` manual
Dono / permissões	Quem cria é o dono; precisa de `CREATE TABLE` no schema e `USE`/`READ VOLUME` na fonte

PASSO 1 — Criar a tabela com agendamento

SQL | Databricks SQL Editor, conectado a um SQL Warehouse, colar tudo e Run | sql/02_streaming_table_b3.sql

```
-- Databricks SQL Editor, conectado a um SQL Warehouse (serverless recomendado).
-- Cria a Streaming Table E agenda uma atualizacao a cada hora, tudo em um comando.
CREATE OR REFRESH STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql
  SCHEDULE EVERY 1 HOUR
AS
SELECT
  *,
  current_timestamp()   AS _ingested_at,
  _metadata.file_path   AS _source_file
FROM STREAM read_files(
  '/Volumes/fininsight_dev/landing/raw/b3/',
  format => 'csv',
  header => true
);

-- Conferir
SELECT * FROM fininsight_dev.bronze.bz_b3_cotacoes_sql LIMIT 10;
```

PASSO 2 — Operar: REFRESH, ALTER SCHEDULE, DROP SCHEDULE

SQL | Databricks SQL Editor, executar os comandos que precisar, um por vez | sql/03_streaming_table_operacao.sql

```
-- Operacao do dia a dia de uma Streaming Table com SCHEDULE (SQL Editor)

-- atualizar agora, fora do agendamento (incremental)
REFRESH STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql;

-- reprocessar tudo do zero (cuidado: le todos os arquivos de novo)
REFRESH STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql FULL;

-- trocar o agendamento para todo dia
ALTER STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql
  ALTER SCHEDULE EVERY 1 DAY;

-- agendamento por CRON (Quartz), 07:00 de segunda a sexta, fuso de Sao Paulo
ALTER STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql
  ALTER SCHEDULE CRON '0 0 7 ? * MON-FRI' AT TIME ZONE 'America/Sao_Paulo';

-- remover o agendamento (a tabela continua existindo; so deixa de atualizar sozinha)
ALTER STREAMING TABLE fininsight_dev.bronze.bz_b3_cotacoes_sql DROP SCHEDULE;

-- ver o que foi criado: pipeline dedicado e agendamento
DESCRIBE EXTENDED fininsight_dev.bronze.bz_b3_cotacoes_sql;
```

Formas da cláusula SCHEDULE

Forma	Exemplo	Observação
Intervalo	`SCHEDULE EVERY 1 HOUR` (também DAY e WEEK)	Mais simples; o valor é inteiro
Cron (Quartz)	`SCHEDULE CRON '0 0 7 ? * MON-FRI' AT TIME ZONE 'America/Sao_Paulo'`	7h de segunda a sexta, fuso explícito
Alterar	`ALTER STREAMING TABLE ... ALTER SCHEDULE ...`	Troca o agendamento sem recriar
Remover	`ALTER STREAMING TABLE ... DROP SCHEDULE`	A tabela fica, só para de atualizar sozinha
Disparo por evento	`TRIGGER ON UPDATE`	Atualiza quando a fonte muda; recurso mais novo, confirme disponibilidade no seu workspace

ATENCAO O `SCHEDULE` vale **somente** para streaming tables standalone. Quando a tabela pertence a um *pipeline* (cap. 21), o agendamento é do pipeline ou de um Job, e `ALTER STREAMING TABLE ... SCHEDULE` não se aplica a ela.

Como acompanhar

- Catalog Explorer, abra a tabela, aba **Refreshes**: histórico de atualizações, duração e erros.
- `DESCRIBE EXTENDED <tabela>` mostra o agendamento e o pipeline gerenciado.
- Jobs & Pipelines lista o job criado pelo `SCHEDULE` (nome começa com o ID da tabela).

CUSTO Cada atualização agendada **liga compute serverless**. `EVERY 1 HOUR` roda 24 vezes ao dia mesmo sem arquivo novo (a execução é curta, mas existe). Para estudo, prefira `EVERY 1 DAY` ou `DROP SCHEDULE` quando terminar.

21. Declarative Pipelines (Lakeflow Spark Declarative Pipelines)

> **NOVO NESTA VERSAO** Capítulo novo. Aqui explicamos o nome e o que você **já construiu**: o pipeline Bronze do cap. 17 é um Declarative Pipeline.

O que são Declarative Pipelines

Lakeflow Spark Declarative Pipelines é o nome atual do Delta Live Tables (DLT). Você escreve **tabelas e views** em Python ou SQL; o Databricks resolve a ordem de execução, o paralelismo, os checkpoints, o retry, as métricas de qualidade e o compute. Um pipeline é o objeto que agrupa os arquivos-fonte.

	Python (from pyspark import pipelines as dp)	SQL
Streaming table	`@dp.table` + `spark.readStream`...	`CREATE OR REFRESH STREAMING TABLE ... AS SELECT ... FROM STREAM ...`
Materialized view	`@dp.materialized_view` + `spark.read`...	`CREATE OR REFRESH MATERIALIZED VIEW ... AS SELECT ...`
Qualidade	`@dp.expect_or_drop(...)`	`CONSTRAINT nome EXPECT (cond) ON VIOLATION DROP ROW`
Arquivo	`pipelines/**/*.py`	`pipelines/**/*.sql`

Streaming table standalone x streaming table em pipeline

	Standalone (cap. 20)	Em pipeline (cap. 17)
Definição	Um comando SQL no editor	Arquivos em uma pasta do Git folder
Agendamento	`SCHEDULE` na própria tabela	Pipeline (botão Schedule) ou Job
Dependências entre tabelas	Você controla na mão	O pipeline monta o grafo sozinho
Versionamento	Só se você guardar o SQL em arquivo	Git + CI/CD naturalmente
Melhor para	Ingestão simples e isolada	Fluxo Bronze, Silver, Gold ponta a ponta

PASSO 1 — Variante em SQL do pipeline Bronze (demonstração)

O arquivo abaixo cria uma streaming table e uma materialized view dentro de um pipeline. **Sem** `SCHEDULE`: o agendamento é do pipeline.

SQL | VS Code, arquivo novo em pipelines/bronze_sql/ | pipelines/bronze_sql/bz_b3_cotacoes_pipeline.sql

```
-- Arquivo de pipeline (Declarative Pipeline em SQL): pipelines/bronze_sql/bz_b3_cotacoes_pipeline.sql
-- Sem SCHEDULE: quem agenda e o pipeline/Job que contem este arquivo.
CREATE OR REFRESH STREAMING TABLE bz_b3_cotacoes_pipe
AS
SELECT
  *,
  current_timestamp()   AS _ingested_at,
  _metadata.file_path   AS _source_file
FROM STREAM read_files(
  '/Volumes/fininsight_dev/landing/raw/b3/',
  format => 'csv',
  header => true
);

-- Materialized View no mesmo arquivo: agregacao recalculada de forma incremental
CREATE OR REFRESH MATERIALIZED VIEW mv_b3_volume_por_dia
AS
SELECT data_pregao, count(*) AS papeis, sum(volume) AS volume_total
FROM bz_b3_cotacoes_pipe
GROUP BY data_pregao;
```

PASSO 2 **Criar um pipeline só para este arquivo**

1. Jobs & Pipelines, Create, ETL pipeline. Nome: `fininsight_bronze_sql_demo`; catálogo `fininsight_dev`; schema `bronze`.
2. Source code: o arquivo `pipelines/bronze_sql/bz_b3_cotacoes_pipeline.sql`. Serverless. Start.
3. Resultado: `bz_b3_cotacoes_pipe` (streaming table) e `mv_b3_volume_por_dia` (materialized view) no schema `bronze`.

PASSO 3 **Agendar um pipeline**

1. Abra o pipeline, botão **Schedule**, **Add schedule**: frequência, fuso horário e notificações. Isso cria um Job com uma pipeline task.
2. Ou, em produção, defina o job no `databricks.yml` (cap. 38), com `pipeline_task` e `schedule`.
3. Modos de execução: *Triggered* (roda e termina; padrão) ou **Continuous** (sempre ligado; mais caro e só para baixa latência).

Operações úteis

Ação	Onde	Efeito
Start	Botão do pipeline	Atualização incremental
Full refresh	Seta ao lado do Start	Reprocessa tudo da fonte (apaga e recarrega as tabelas selecionadas)
Refresh selected	Seleção de tabelas no grafo	Atualiza só algumas tabelas
Event log	`event_log(TABLE(catalogo.schema.tabela))`	Métricas, expectations e erros

22. Comparativo: COPY INTO x Auto Loader x Streaming Table x Pipeline

O mesmo conjunto de arquivos da B3 pode entrar na Bronze por quatro caminhos. O projeto mantém **quatro tabelas** para você comparar na prática:

Tabela	Como foi criada	Cap.
`bz_b3_cotacoes`	Auto Loader dentro do pipeline Python (a oficial, alimenta a Silver)	17
`bz_b3_cotacoes_copyinto`	COPY INTO (SQL)	19
`bz_b3_cotacoes_sql`	Streaming table standalone com SCHEDULE	20
`bz_b3_cotacoes_pipe`	Streaming table em pipeline SQL	21

Quadro comparativo

Critério	COPY INTO	Auto Loader (pipeline Python)	Streaming Table standalone	Streaming Table em pipeline SQL
Linguagem	SQL	Python (ou SQL)	SQL	SQL
Idempotência	Sim (arquivos já carregados)	Sim (checkpoint)	Sim (checkpoint)	Sim (checkpoint)
Estado	Sem streaming; lista de arquivos	Checkpoint gerenciado	Checkpoint gerenciado	Checkpoint gerenciado
Schema evolution	`mergeSchema`	`schemaEvolutionMode`	Via `read_files` (modo padrão `addNewColumns`)	Idem
Escala de arquivos	Milhares	Milhões	Milhões	Milhões
Agendamento	Você agenda (Job/notebook)	Pipeline/Job	`SCHEDULE` embutido	Pipeline/Job
Compute	O que executar o comando	Pipeline (serverless/classic)	Pipeline serverless dedicado	Pipeline
Dependências/linhagem	Manual	Grafo automático	Manual	Grafo automático
Qualidade (expectations)	Não	Sim	Limitado	Sim
Versionamento em Git	Script SQL	Naturalmente	Script SQL	Naturalmente

Qual escolher

- **Carga pontual ou de baixo volume, em SQL puro** (ex.: carga de um dump): COPY INTO.
- **Fluxo completo Bronze, Silver, Gold com qualidade e linhagem** (nosso caso): Auto Loader dentro de um pipeline declarativo.
- **Uma tabela isolada que precisa atualizar sozinha, sem montar projeto**: streaming table standalone com SCHEDULE.
- **Pipeline escrito em SQL por equipe de analytics**: streaming tables e materialized views em arquivos `.sql` de pipeline.
- **Dado em tempo real (Kafka/Event Hubs)**: streaming table (Python ou SQL) lendo `kafka`, como no cap. 23.

> **DICA** Regra prática do projeto: a Silver e a Gold só leem `bz_b3_cotacoes` (Auto Loader). As outras três são para estudo e comparação de custo/comportamento; apague-as quando terminar (`DROP TABLE`), principalmente a que tem `SCHEDULE`.

23. Streaming com Event Hubs (Kafka)

Para simular dados em tempo real, usamos o **Azure Event Hubs**, que expõe um endpoint compatível com **Kafka** (porta 9093). O Databricks lê com `spark.readStream.format("kafka")` sem nenhuma biblioteca extra.

> **CUSTO** O Event Hubs Standard **cobra por hora enquanto o namespace existir**, mesmo sem tráfego. Aplique só quando for usar e remova depois com `terraform destroy -target=module.eventhubs`.

PASSO 1 — Terraform: ativar o módulo eventhubs

O módulo cria o namespace (tier Standard, porque o Basic **não** suporta Kafka), o hub `cotacoes-tempo-real`, uma regra de acesso e, no scope `fininsight` (criado pelo notebook 00 no cap. 14), os secrets `eventhub-connection-string` e `eventhub-bootstrap`. Crie os quatro arquivos abaixo em `infra/modules/eventhubs/` e, em `infra/main.tf`, **remova o** `#` do bloco `module "eventhubs"`.

TERRAFORM | VS Code, arquivo novo | infra/modules/eventhubs/versions.tf

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
    }
    databricks = {
      source  = "databricks/databricks"
      version = "~> 1.50"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/eventhubs/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}

variable "secret_scope" {
  description = "Scope de secrets do Databricks (criado pelo notebook 00)"
  type        = string
  default     = "fininsight"
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/eventhubs/main.tf

```
resource "random_string" "suffix" {
  length  = 4
  upper   = false
  special = false
}

# Kafka endpoint exige o tier Standard (Basic nao suporta)
resource "azurerm_eventhub_namespace" "ns" {
  name                = "evhns-fininsight-${var.environment}-${random_string.suffix.result}"
  location            = var.location
  resource_group_name = var.resource_group_name
  sku                 = "Standard"
  capacity            = 1
}

resource "azurerm_eventhub" "cotacoes" {
  name                = "cotacoes-tempo-real"
  namespace_name      = azurerm_eventhub_namespace.ns.name
  resource_group_name = var.resource_group_name
```

TERRAFORM | VS Code, arquivo novo | infra/modules/eventhubs/main.tf

```
  partition_count     = 2
  message_retention   = 1
}

resource "azurerm_eventhub_namespace_authorization_rule" "databricks" {
  name                = "databricks"
  namespace_name      = azurerm_eventhub_namespace.ns.name
  resource_group_name = var.resource_group_name
  listen              = true
  send                = true
  manage              = false
}

# Secrets lidos pelo producer e pelo pipeline. O scope "fininsight" e criado pelo
# notebook 00_configurar_secrets.py (ele precisa existir antes deste apply).
resource "databricks_secret" "eventhub_connection_string" {
  scope        = var.secret_scope
  key          = "eventhub-connection-string"
  string_value = azurerm_eventhub_namespace_authorization_rule.databricks.primary_connection_string
}

resource "databricks_secret" "eventhub_bootstrap" {
  scope        = var.secret_scope
  key          = "eventhub-bootstrap"
  string_value = "${azurerm_eventhub_namespace.ns.name}.servicebus.windows.net:9093"
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/eventhubs/outputs.tf

```
output "namespace_name" {
  value = azurerm_eventhub_namespace.ns.name
}
```

TERMINAL | VS Code | dentro de infra/ | init necessário (novo provider random)

```
cd infra
terraform init
terraform plan -var-file="envs/dev/terraform.tfvars" -out=plan.out
terraform apply plan.out
```

PASSO 2 Conferir os secrets

PYTHON | Databricks, célula de notebook | verificação

```
print([s.key for s in dbutils.secrets.list("fininsight")])
# esperado: ['eventhub-bootstrap', 'eventhub-connection-string'] (mais as chaves de API, se ja
    configuradas)
```

PASSO 3 Producer: enviar cotações simuladas

Gera lotes de cotações sintéticas (5 ativos, 60 lotes a cada 5 segundos) e os envia ao tópico via protocolo Kafka.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/02_streaming_producer.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 02 - Producer de cotações simuladas (Event Hubs via protocolo Kafka)
# MAGIC Envia lotes de cotações sintéticas para o tópico `cotacoes-tempo-real`.

# COMMAND ----------

import json
import random
import time
from datetime import datetime, timezone

SCOPE = "fininsight"
TOPICO = "cotacoes-tempo-real"
RODADAS = 60          # quantidade de lotes
INTERVALO_S = 5       # segundos entre lotes

bootstrap = dbutils.secrets.get(SCOPE, "eventhub-bootstrap")
conexao = dbutils.secrets.get(SCOPE, "eventhub-connection-string")
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/02_streaming_producer.py

```
kafka_opts = {
    "kafka.bootstrap.servers": bootstrap,
    "kafka.security.protocol": "SASL_SSL",
    "kafka.sasl.mechanism": "PLAIN",
    "kafka.sasl.jaas.config": (
        "kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required "
        f'username="$ConnectionString" password="{conexao}";'
    ),
}

precos = {"PETR4.SA": 38.0, "VALE3.SA": 62.0, "ITUB4.SA": 33.0, "AAPL": 230.0, "MSFT": 420.0}

# COMMAND ----------

for rodada in range(RODADAS):
    linhas = []
    for ticker, p in precos.items():
        precos[ticker] = round(p * (1 + random.gauss(0, 0.002)), 4)
        evento = {
            "ticker": ticker,
            "preco": precos[ticker],
            "volume": random.randint(100, 5000),
            "ts": datetime.now(timezone.utc).isoformat(),
        }
        linhas.append((ticker, json.dumps(evento)))

    (spark.createDataFrame(linhas, ["key", "value"])
          .write.format("kafka")
          .options(**kafka_opts)
          .option("topic", TOPICO)
          .save())
    print(f"lote {rodada + 1}/{RODADAS} enviado")
    time.sleep(INTERVALO_S)
```

PASSO 4 Tabela Bronze de streaming

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/bronze/ (8º arquivo do pipeline fininsight_bronze) | pipelines/bronze/bz_cotacoes_streaming.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import current_timestamp

SCOPE = "fininsight"
TOPICO = "cotacoes-tempo-real"


def _opcoes_kafka():
    bootstrap = dbutils.secrets.get(SCOPE, "eventhub-bootstrap")
    conexao = dbutils.secrets.get(SCOPE, "eventhub-connection-string")
    return {
        "kafka.bootstrap.servers": bootstrap,
        "kafka.security.protocol": "SASL_SSL",
        "kafka.sasl.mechanism": "PLAIN",
        "kafka.sasl.jaas.config": (
            "kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required "
            f'username="$ConnectionString" password="{conexao}";'
        ),
        "subscribe": TOPICO,
        "startingOffsets": "earliest",
    }


@dp.table(
    name="bz_cotacoes_streaming",
    comment="Cotacoes simuladas em tempo real via Event Hubs (raw)",
    table_properties={"quality": "bronze"},
)
def bz_cotacoes_streaming():
    return (
        spark.readStream.format("kafka")
        .options(**_opcoes_kafka())
        .load()
        .selectExpr("CAST(value AS STRING) AS json_payload", "timestamp AS kafka_ts")
        .withColumn("_ingested_at", current_timestamp())
    )
```

CORRIGIDO Erro `eventhub_bootstrap_servers` indefinido na v1 (variável usada sem ser declarada): agora tudo vem de `dbutils.secrets.get` dentro de `_opcoes_kafka()`.

Reinicie o pipeline `fininsight_bronze` (agora com os 8 arquivos). `bz_cotacoes_streaming` guarda o JSON cru em `json_payload` e o horário do Kafka em `kafka_ts`; o parse do JSON (ticker, preço, volume) é feito adiante, na Silver, mantendo a Bronze fiel ao que chegou.

SQL | Databricks SQL Editor | conferir o streaming

```
SELECT kafka_ts,
       json_payload,
       from_json(json_payload, 'ticker STRING, preco DOUBLE, volume INT, ts STRING') AS evento
FROM fininsight_dev.bronze.bz_cotacoes_streaming
ORDER BY kafka_ts DESC
LIMIT 20;
```

Opção Kafka	Valor	Motivo
`kafka.bootstrap.servers`	`<namespace>.servicebus.windows.net:9093`	Endpoint Kafka do Event Hubs
`kafka.security.protocol` / `sasl.mechanism`	`SASL_SSL` / `PLAIN`	Autenticação por connection string
`username`	`$ConnectionString` (literal)	Convenção do Event Hubs; a senha é a connection string
`PlainLoginModule` com prefixo `kafkashaded.`	-	No Databricks as classes Kafka são empacotadas com esse prefixo
`startingOffsets`	`earliest`	Lê o tópico desde o início na primeira vez

PONTO DE VERIFICACAO **Fim da Sprint 3: formas de ingestão e streaming**

- Você sabe criar uma tabela Bronze por Auto Loader (pipeline), COPY INTO, Streaming Table com `SCHEDULE` e pipeline SQL, e escolher entre elas;
- Event Hubs aplicado, producer enviando lotes e `bz_cotacoes_streaming` com linhas (se ativou o streaming);
- Tabelas de demonstração (`_copyinto`, `_sql`, `_pipe`) apagadas ou com agendamento removido para não gerar custo.

Parte V - Sprint 4: Camada Silver

24. Delta Lake na prática

Todas as tabelas das três camadas são **Delta**. Este capítulo mostra, com comandos que você executa, o que o formato oferece. Os conceitos estão resumidos no cap. 3; aqui é a prática.

> **NOTA** Tabelas criadas por pipelines declarativos são **gerenciadas**: o Databricks faz OPTIMIZE e VACUUM sozinho (otimização preditiva). Por isso o script cria uma **cópia de demonstração** (`demo_delta`) para você praticar a manutenção manual sem mexer na Bronze.

SQL | Databricks SQL Editor, executar bloco a bloco e observar o resultado | sql/05_delta_lake_demo.sql

```
-- SQL Editor (warehouse) ou celulas %sql.
-- Tabela usada: bronze.bz_b3_cotacoes (criada pelo pipeline Bronze).

-- 1) Historico de operacoes (cada linha = uma versao da tabela)
DESCRIBE HISTORY fininsight_dev.bronze.bz_b3_cotacoes;

-- 2) Time travel por versao ou por data
SELECT count(*) FROM fininsight_dev.bronze.bz_b3_cotacoes VERSION AS OF 0;
SELECT count(*) FROM fininsight_dev.bronze.bz_b3_cotacoes TIMESTAMP AS OF '2026-10-01T00:00:00Z';

-- 3) Detalhes fisicos: formato, arquivos, tamanho, localizacao
DESCRIBE DETAIL fininsight_dev.bronze.bz_b3_cotacoes;

-- 4) Tabelas MANAGED pelo pipeline: a manutencao (OPTIMIZE/VACUUM) e automatica.
--    Em tabelas Delta suas (ex.: copia de demonstracao) voce roda manualmente:
CREATE OR REPLACE TABLE fininsight_dev.bronze.demo_delta AS
SELECT * FROM fininsight_dev.bronze.bz_b3_cotacoes;

OPTIMIZE fininsight_dev.bronze.demo_delta ZORDER BY (codigo, data_pregao);
VACUUM fininsight_dev.bronze.demo_delta RETAIN 168 HOURS;

-- 5) MERGE (upsert): corrige/insere linhas de forma atomica
MERGE INTO fininsight_dev.bronze.demo_delta AS alvo
USING (SELECT 'PETR4' AS codigo, DATE'2026-10-02' AS data_pregao, 99.99 AS preco_fechamento) AS fonte
  ON alvo.codigo = fonte.codigo AND alvo.data_pregao = fonte.data_pregao
WHEN MATCHED THEN UPDATE SET alvo.preco_fechamento = fonte.preco_fechamento
WHEN NOT MATCHED THEN INSERT (codigo, data_pregao, preco_fechamento)
  VALUES (fonte.codigo, fonte.data_pregao, fonte.preco_fechamento);

-- 6) Desfazer: voltar a tabela para a versao anterior ao MERGE
DESCRIBE HISTORY fininsight_dev.bronze.demo_delta;
RESTORE TABLE fininsight_dev.bronze.demo_delta TO VERSION AS OF 1;

-- 7) Limpeza da demonstracao
DROP TABLE IF EXISTS fininsight_dev.bronze.demo_delta;
```

Comando	O que mostra
`DESCRIBE HISTORY`	Uma linha por versão: operação, usuário, métricas (arquivos e linhas gravadas)
`VERSION AS OF` / `TIMESTAMP AS OF`	Lê a tabela como era; base para auditoria e depuração de KPI
`DESCRIBE DETAIL`	Local no storage, número de arquivos, tamanho
`OPTIMIZE ... ZORDER BY`	Compacta arquivos pequenos e agrupa dados pelas colunas de filtro
`VACUUM ... RETAIN 168 HOURS`	Apaga arquivos antigos sem referência (7 dias); limita o time travel
`MERGE INTO`	Upsert atômico; é o mecanismo por trás do CDC (cap. 28)
`RESTORE TABLE`	Volta a tabela a uma versão anterior (desfaz um erro)

25. Silver: padronização e deduplicação

Cada camada tem seu pipeline. O pipeline `fininsight_silver` lê as tabelas Bronze por nome completo (`fininsight_dev.bronze.<tabela>`) e escreve em `fininsight_dev.silver`.

NOVO NESTA VERSAO Mudança de arquitetura: **um pipeline por camada** (bronze, silver, gold) em vez de um pipeline único. Cada camada pode ser atualizada, reiniciada e depurada separadamente, e o job do cap. 38 encadeia os três.

NOTA As tabelas da Silver são **materialized views** (leem com `spark.read`, não `readStream`), porque funções de janela com `lag` e médias móveis não são suportadas em streaming. O catálogo vem da configuração `fininsight.catalog` do pipeline (padrão `fininsight_dev`).

PASSO 1 **Criar os arquivos em pipelines/silver/**

sv_precos_ativos: unificação B3 + yfinance e deduplicação

O COTAHIST traz centenas de papéis: filtramos a **watchlist** e padronizamos o ticker (`PETR4` vira `PETR4.SA`). Se a mesma ação/data vem das duas fontes, fica a da B3 (prioridade 1); `row_number()` faz a deduplicação.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_precos_ativos.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")
WATCHLIST = ["PETR4.SA", "VALE3.SA", "ITUB4.SA"]   # tickers da B3 que acompanhamos


@dp.materialized_view(
    name="sv_precos_ativos",
    comment="Precos diarios unificados (B3 + yfinance), deduplicados por ticker/data",
    table_properties={"quality": "silver"},
)
@dp.expect_or_drop("preco_valido", "preco_fechamento > 0")
@dp.expect_or_drop("ticker_valido", "ticker IS NOT NULL")
def sv_precos_ativos():
    # B3: o COTAHIST traz centenas de papeis; filtramos a watchlist e padronizamos o ticker
    b3 = (
        spark.read.table(f"{CAT}.bronze.bz_b3_cotacoes")
        .withColumn("ticker", F.concat(F.trim(F.col("codigo")), F.lit(".SA")))
        .filter(F.col("ticker").isin(WATCHLIST))
        .select(
            "ticker",
            F.to_date("data_pregao").alias("data_pregao"),
            F.col("preco_fechamento").cast("double").alias("preco_fechamento"),
            F.col("volume").cast("double").alias("volume"),
            F.lit("B3").alias("origem"),
            F.lit(1).alias("prioridade"),
            "_ingested_at",
        )
    )
    yf = (
        spark.read.table(f"{CAT}.bronze.bz_yfinance_precos")
        .select(
            "ticker",
            F.to_date("Date").alias("data_pregao"),
            F.col("Close").cast("double").alias("preco_fechamento"),
            F.col("Volume").cast("double").alias("volume"),
            F.lit("YFINANCE").alias("origem"),
            F.lit(2).alias("prioridade"),
            "_ingested_at",
        )
    )
    # mesma acao/data em duas fontes -> fica a de maior prioridade (B3), depois a mais recente
    janela = Window.partitionBy("ticker", "data_pregao").orderBy(
        F.col("prioridade"), F.col("_ingested_at").desc()
    )
    return (
        b3.unionByName(yf)
        .withColumn("rn", F.row_number().over(janela))
        .filter("rn = 1")
```

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_precos_ativos.py

```
        .drop("rn", "prioridade")
    )
```

sv_indicadores_macro: Selic/IPCA (BCB) e FRED em formato longo

Usamos `try_cast` porque o FRED devolve `"."` quando não há dado, e um `cast` comum quebraria o pipeline (modo ANSI).

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_indicadores_macro.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="sv_indicadores_macro",
    comment="Selic/IPCA (BCB) e Fed Funds/CPI/10Y (FRED) em formato longo",
    table_properties={"quality": "silver"},
)
@dp.expect_or_drop("valor_numerico", "valor IS NOT NULL")
def sv_indicadores_macro():
    bcb = (
        spark.read.table(f"{CAT}.bronze.bz_bcb_series")
        .filter(F.col("serie").isin("selic", "ipca"))
        .select(
            F.col("serie").alias("indicador"),
            F.to_date("data", "dd/MM/yyyy").alias("data_referencia"),
            F.expr("try_cast(valor AS DOUBLE)").alias("valor"),
            F.lit("BCB").alias("fonte"),
        )
    )
    fred = (
        spark.read.table(f"{CAT}.bronze.bz_fred_indicadores")
        .select(
            F.lower(F.col("serie")).alias("indicador"),
            F.to_date("data").alias("data_referencia"),
            F.expr("try_cast(valor AS DOUBLE)").alias("valor"),   # FRED usa "." para dado ausente
            F.lit("FRED").alias("fonte"),
        )
    )
    return bcb.unionByName(fred).dropDuplicates(["indicador", "data_referencia"])
```

sv_cambio: dólar do BCB e do Alpha Vantage

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_cambio.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="sv_cambio",
    comment="Dolar diario: serie SGS 1 do BCB e fechamento USD/BRL do Alpha Vantage",
    table_properties={"quality": "silver"},
)
@dp.expect_or_drop("ptax_positivo", "ptax_venda > 0")
def sv_cambio():
    ptax = (
        spark.read.table(f"{CAT}.bronze.bz_bcb_series")
        .filter(F.col("serie") == "ptax_dolar")
        .select(
            F.to_date("data", "dd/MM/yyyy").alias("data_cotacao"),
            F.expr("try_cast(valor AS DOUBLE)").alias("ptax_venda"),
        )
        .dropDuplicates(["data_cotacao"])
    )
    av = (
        spark.read.table(f"{CAT}.bronze.bz_alphavantage_fx")
        .select(
            F.to_date("data").alias("data_cotacao"),
            F.col("close").cast("double").alias("usdbrl_close"),
        )
        .dropDuplicates(["data_cotacao"])
```

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_cambio.py

```
    )
    return ptax.join(av, "data_cotacao", "full_outer").filter("ptax_venda IS NOT NULL")
```

sv_cripto: última cotação do dia por criptoativo

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_cripto.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="sv_cripto",
    comment="Ultima cotacao de cada criptoativo por dia (USD)",
    table_properties={"quality": "silver"},
)
@dp.expect_or_drop("preco_positivo", "preco_usd > 0")
def sv_cripto():
    df = (
        spark.read.table(f"{CAT}.bronze.bz_coingecko_cripto")
        .withColumn("coletado_ts", F.to_timestamp("coletado_em"))
        .withColumn("data_referencia", F.to_date("coletado_ts"))
        .select(
            F.col("id").alias("ativo_id"),
            F.upper("simbolo").alias("simbolo"),
            "data_referencia",
            "coletado_ts",
            F.col("preco_usd").cast("double").alias("preco_usd"),
            F.col("market_cap").cast("double").alias("market_cap"),
            F.col("volume_24h").cast("double").alias("volume_24h"),
        )
    )
    janela = Window.partitionBy("ativo_id", "data_referencia").orderBy(F.col("coletado_ts").desc())
    return df.withColumn("rn", F.row_number().over(janela)).filter("rn = 1").drop("rn", "coletado_ts")
```

sv_noticias_processadas: limpeza de texto e deduplicação

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_noticias_processadas.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="sv_noticias_processadas",
    comment="Noticias limpas (sem HTML/espacos extras), deduplicadas por noticia_id",
    table_properties={"quality": "silver"},
)
@dp.expect_or_drop("texto_presente", "texto_noticia IS NOT NULL AND length(texto_noticia) > 20")
@dp.expect("ticker_identificado", "ticker_relacionado IS NOT NULL")   # apenas mede, nao descarta
def sv_noticias_processadas():
    limpo = F.trim(F.regexp_replace(F.regexp_replace("texto_noticia", "<[^>]*>", " "), "\\s+", " "))
    return (
        spark.read.table(f"{CAT}.bronze.bz_noticias_raw")
        .withColumn("texto_noticia", limpo)
        .withColumn("data_publicacao_ts", F.to_timestamp("data_publicacao"))
        .withColumn("data_publicacao", F.to_date("data_publicacao_ts"))
        .withColumn("qtd_palavras", F.size(F.split("texto_noticia", " ")))
        .select("noticia_id", "titulo", "texto_noticia", "fonte", "data_publicacao",
                "data_publicacao_ts", "url", "ticker_relacionado", "qtd_palavras")
        .dropDuplicates(["noticia_id"])
    )
```

26. Silver: cálculos financeiros (retorno, SMA, RSI, z-score)

Aqui está o coração da plataforma: a Silver **calcula**, não apenas limpa. Tudo é feito com **window functions** particionadas por `ticker` e ordenadas por data.

Métrica	Fórmula	Janela
Retorno logarítmico	ln(P_t / P_t-1)	lag de 1 dia
SMA 21d / 63d	média simples dos preços	21 / 63 pregões
Volatilidade anualizada	desvio-padrão dos retornos x raiz(252)	21 pregões
RSI 14d	100 - 100 / (1 + média dos ganhos / média das perdas)	14 pregões
Z-score do preço	(P - SMA63) / desvio63	63 pregões

> **DICA** Por que retorno **logarítmico**? Ele é aditivo no tempo (a soma dos retornos diários é o retorno do período), simétrico e é a base de volatilidade e VaR.

PASSO 1 Criar o arquivo

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_retornos_volatilidade.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window


@dp.materialized_view(
    name="sv_retornos_volatilidade",
    comment="Retorno log, medias moveis, volatilidade anualizada, RSI e z-score por ticker/dia",
    table_properties={"quality": "silver"},
)
def sv_retornos_volatilidade():
    df = spark.read.table("sv_precos_ativos")

    tempo = Window.partitionBy("ticker").orderBy("data_pregao")
    j14 = tempo.rowsBetween(-13, 0)
    j21 = tempo.rowsBetween(-20, 0)
    j63 = tempo.rowsBetween(-62, 0)

    # retorno logaritmico diario: ln(P_t / P_t-1) -- aditivo no tempo, padrao em financas
    df = df.withColumn("preco_anterior", F.lag("preco_fechamento", 1).over(tempo))
    df = df.withColumn(
        "retorno_log_1d",
        F.when(F.col("preco_anterior") > 0, F.log(F.col("preco_fechamento") / F.col("preco_anterior"))),
    )

    # medias moveis simples
    df = df.withColumn("sma_21d", F.avg("preco_fechamento").over(j21))
    df = df.withColumn("sma_63d", F.avg("preco_fechamento").over(j63))

    # volatilidade anualizada: desvio-padrao dos retornos diarios * raiz(252 pregoes)
    df = df.withColumn("vol_21d_anualizada", F.stddev("retorno_log_1d").over(j21) * F.lit(252 ** 0.5))

    # RSI de 14 dias (media simples de ganhos e perdas)
    ret = F.col("retorno_log_1d")
    df = df.withColumn("ganho", F.when(ret > 0, ret).otherwise(0.0))
    df = df.withColumn("perda", F.when(ret < 0, -ret).otherwise(0.0))
    df = df.withColumn("media_ganho_14d", F.avg("ganho").over(j14))
    df = df.withColumn("media_perda_14d", F.avg("perda").over(j14))
    df = df.withColumn(
        "rsi_14d",
        F.when(F.col("media_perda_14d") == 0, F.lit(100.0)).otherwise(
            100 - (100 / (1 + F.col("media_ganho_14d") / F.col("media_perda_14d")))
        ),
    )

    # z-score do preco na janela de 63 dias: (preco - media) / desvio
    df = df.withColumn("std_63d", F.stddev("preco_fechamento").over(j63))
    df = df.withColumn(
        "zscore_preco_63d",
        F.when(F.col("std_63d") > 0, (F.col("preco_fechamento") - F.col("sma_63d")) / F.col("std_63d")),
    )
```

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_retornos_volatilidade.py

```
    return df.drop("ganho", "perda", "preco_anterior", "media_ganho_14d", "media_perda_14d", "std_63d")
```

PASSO 2 Conferir com números que você calcula na mão

`tests/test_formulas_financeiras.py` traz as mesmas fórmulas em pandas, com casos que você confere de cabeça (soma dos retornos log, drawdown de 25%, VaR de uma volatilidade conhecida). Depois de rodar o pipeline, compare o Spark com a referência:

SQL | Databricks SQL Editor | depois que o pipeline Silver rodar

```
SELECT ticker, data_pregao, preco_fechamento, retorno_log_1d, sma_21d, vol_21d_anualizada, rsi_14d,
    zscore_preco_63d
FROM fininsight_dev.silver.sv_retornos_volatilidade
WHERE ticker = 'PETR4.SA'
ORDER BY data_pregao DESC
LIMIT 30;
```

ATENCAO Sem histórico (cap. 15, backfill de 2 anos) as janelas ficam incompletas: SMA de 63 dias vira média de poucos pontos e `vol_21d_anualizada` é nula nas primeiras linhas. Isso é esperado, não é bug.

PASSO 3 Rodar a bateria de testes de referência (opcional)

TERMINAL | VS Code | raiz do repositório

```
pip install pytest pandas
pytest tests -q
```

27. Silver: join multi-fonte e o pipeline Silver

sv_precos_com_macro: juntando preço, dólar e Selic

Une três fontes pela data do pregão: preços com indicadores (BRL), PTAX do dia e Selic diária.

> **CORRIGIDO** Duas correções de finanças: (1) a Selic da série 11 do BCB vem em **% ao dia** (0,055 = 0,055%), então a taxa livre de risco anual é `selic/100 * 252`; a v1 multiplicava por 252 sem dividir por 100 e inflava o resultado 100 vezes. (2) O BCB entrega só os últimos 20 pontos por chamada; onde não há Selic do dia, usamos a média disponível como aproximação documentada.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_precos_com_macro.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window


@dp.materialized_view(
    name="sv_precos_com_macro",
    comment="Precos + indicadores tecnicos + dolar (PTAX) + Selic diaria: join multi-fonte",
    table_properties={"quality": "silver"},
)
def sv_precos_com_macro():
    precos = spark.read.table("sv_retornos_volatilidade")

    cambio = spark.read.table("sv_cambio").select(
        F.col("data_cotacao").alias("data_pregao"), "ptax_venda"
    )

    # SGS 11 = Selic em % ao dia (ex.: 0.055 = 0,055% a.d.)
    selic = (
        spark.read.table("sv_indicadores_macro")
        .filter("indicador = 'selic'")
        .select(F.col("data_referencia").alias("data_pregao"), F.col("valor").alias("selic_diaria_pct"))
    )

    df = precos.join(cambio, "data_pregao", "left").join(selic, "data_pregao", "left")

    # O BCB so entrega os ultimos 20 pontos por chamada: onde nao ha Selic do dia usamos a
    # media da Selic disponivel como taxa livre de risco (aproximacao documentada)
    media_selic = F.avg("selic_diaria_pct").over(Window.partitionBy(F.lit(1)))
    return (
        df.withColumn("rf_diaria_pct", F.coalesce("selic_diaria_pct", media_selic))
        .withColumn("preco_fechamento_usd", F.col("preco_fechamento") / F.col("ptax_venda"))
    )
```

Criar o pipeline Silver

1. Jobs & Pipelines, Create, ETL pipeline. Nome: `fininsight_silver`. **Default catalog** `fininsight_dev`, **Default schema** `silver`.
2. Source code: a **pasta** `pipelines/silver` (inclui `sv_carteiras_scd2.py`, que depende do cap. 28; se ainda não criou a tabela de CDC, exclua esse arquivo por enquanto).
3. Em **Advanced, Configuration** adicione a chave `fininsight.catalog` com valor `fininsight_dev`.
4. Serverless, modo Development, **Start**. O grafo mostra Bronze (leitura externa) alimentando as tabelas Silver.

SQL | Databricks SQL Editor | verificação da Silver

```
SELECT 'sv_precos_ativos' AS tabela, count(*) AS linhas FROM fininsight_dev.silver.sv_precos_ativos
UNION ALL SELECT 'sv_retornos_volatilidade', count(*) FROM fininsight_dev.silver.sv_retornos_volatilidade
UNION ALL SELECT 'sv_precos_com_macro', count(*) FROM fininsight_dev.silver.sv_precos_com_macro
UNION ALL SELECT 'sv_indicadores_macro', count(*) FROM fininsight_dev.silver.sv_indicadores_macro
UNION ALL SELECT 'sv_cambio', count(*) FROM fininsight_dev.silver.sv_cambio
UNION ALL SELECT 'sv_cripto', count(*) FROM fininsight_dev.silver.sv_cripto
UNION ALL SELECT 'sv_noticias_processadas', count(*) FROM fininsight_dev.silver.sv_noticias_processadas;
```

28. CDC e SCD Type 2

CDC (Change Data Capture) é receber *eventos de mudança* (inserir, alterar, apagar) em vez do estado final. **SCD Type 2** guarda o histórico: cada mudança cria uma nova linha com validade (`__START_AT` e `__END_AT`). Aqui aplicamos ao histórico de composição das carteiras.

PASSO 1 Criar a tabela de eventos (fonte do CDC)

SQL | Databricks SQL Editor, colar tudo e Run | sql/04_seed_carteiras_cdc.sql

```
-- SQL Editor. Cria a tabela de eventos de CDC usada pelo SCD Type 2 (sv_carteiras_scd2).
CREATE TABLE IF NOT EXISTS fininsight_dev.bronze.bz_composicao_carteiras_cdc (
  carteira_id STRING,
  ticker      STRING,
  peso        DOUBLE,
  data_evento TIMESTAMP,
  operacao    STRING      -- UPSERT ou DELETE
);

INSERT INTO fininsight_dev.bronze.bz_composicao_carteiras_cdc VALUES
  ('conservadora', 'PETR4.SA', 0.30, TIMESTAMP'2026-01-02 09:00:00', 'UPSERT'),
  ('conservadora', 'ITUB4.SA', 0.70, TIMESTAMP'2026-01-02 09:00:00', 'UPSERT'),
  ('conservadora', 'PETR4.SA', 0.20, TIMESTAMP'2026-03-01 09:00:00', 'UPSERT'),  -- rebalanceamento
  ('conservadora', 'VALE3.SA', 0.10, TIMESTAMP'2026-03-01 09:00:00', 'UPSERT'),
  ('conservadora', 'ITUB4.SA', 0.70, TIMESTAMP'2026-06-01 09:00:00', 'DELETE'),  -- saiu da carteira
  ('agressiva',    'AAPL',     0.50, TIMESTAMP'2026-01-02 09:00:00', 'UPSERT'),
  ('agressiva',    'MSFT',     0.50, TIMESTAMP'2026-01-02 09:00:00', 'UPSERT');
```

PASSO 2 Criar o arquivo do pipeline

`create_auto_cdc_flow` lê os eventos, ordena por `data_evento` (`sequence_by`), aplica `DELETE` quando `operacao = 'DELETE'` e mantém o histórico (`stored_as_scd_type=2`).

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/silver/ | pipelines/silver/sv_carteiras_scd2.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.temporary_view(name="vw_carteiras_cdc")
def vw_carteiras_cdc():
    # fonte de eventos de CDC: tabela Delta criada por sql/04_seed_carteiras_cdc.sql
    return spark.readStream.table(f"{CAT}.bronze.bz_composicao_carteiras_cdc")


# Tabela-alvo: historico versionado (SCD Type 2) da composicao das carteiras
dp.create_streaming_table(
    name="sv_carteiras_scd2",
    comment="Composicao historica das carteiras (SCD2: colunas __START_AT / __END_AT)",
    table_properties={"quality": "silver"},
)

dp.create_auto_cdc_flow(
    target="sv_carteiras_scd2",
    source="vw_carteiras_cdc",
    keys=["carteira_id", "ticker"],
    sequence_by=F.col("data_evento"),
    apply_as_deletes=F.expr("operacao = 'DELETE'"),
    except_column_list=["operacao"],
    stored_as_scd_type=2,
)
```

PASSO 3 Rodar o pipeline Silver e consultar

SQL | Databricks SQL Editor | consulta point-in-time

```
-- historico completo
SELECT * FROM fininsight_dev.silver.sv_carteiras_scd2 ORDER BY carteira_id, ticker, __START_AT;

-- como era a carteira conservadora em 15/02/2026?
SELECT carteira_id, ticker, peso
```

SQL | Databricks SQL Editor | consulta point-in-time

```
FROM fininsight_dev.silver.sv_carteiras_scd2
WHERE carteira_id = 'conservadora'
  AND __START_AT <= TIMESTAMP'2026-02-15'
  AND (__END_AT IS NULL OR __END_AT > TIMESTAMP'2026-02-15');
```

> **NOTA** Resultado esperado: PETR4 aparece duas vezes (peso 0,30 até 01/03 e 0,20 depois); ITUB4 tem `__END_AT` em 01/06 (saiu da carteira). Na API antiga (`import dlt`) o mesmo comando chamava-se `dlt.apply_changes`; hoje é `dp.create_auto_cdc_flow`.

29. Expectations e qualidade de dados

Diretiva	Comportamento	Uso na FinInsight
`expect`	Registra a métrica e mantém a linha	`ticker_identificado` nas notícias; `sharpe_calculado` na Gold
`expect_or_drop`	Descarta a linha reprovada	`preco_fechamento > 0`, `ticker IS NOT NULL`
`expect_or_fail`	Interrompe a atualização inteira	Chave nula na fato de risco

Já aplicamos expectations nos arquivos dos capítulos 25 a 27 e 33. As regras são decoradores da função, acima do `def`:

PIPELINE DECLARATIVO (PYTHON) | trecho ilustrativo (o arquivo completo está no cap. 25) | pipelines/silver/sv_precos_ativos.py

```
@dp.materialized_view(name="sv_precos_ativos", table_properties={"quality": "silver"})
@dp.expect_or_drop("preco_valido", "preco_fechamento > 0")
@dp.expect_or_drop("ticker_valido", "ticker IS NOT NULL")
def sv_precos_ativos():
    ...  # corpo completo no cap. 25
```

Ver as métricas

Na tela do pipeline, clique na tabela e abra a aba **Data quality**. Por SQL, leia o event log:

SQL | Databricks SQL Editor | sql/06_event_log_qualidade.sql

```
-- SQL Editor. Metricas de expectations (qualidade de dados) de uma tabela do pipeline Silver.
-- Troque a tabela pela que quiser inspecionar.
WITH eventos AS (
  SELECT timestamp, details
  FROM event_log(TABLE(fininsight_dev.silver.sv_precos_ativos))
  WHERE event_type = 'flow_progress'
)
SELECT
  timestamp,
  exp.name                AS expectation,
  exp.dataset             AS tabela,
  exp.passed_records      AS aprovados,
  exp.failed_records      AS reprovados
FROM eventos
LATERAL VIEW explode(
  from_json(details:flow_progress.data_quality.expectations,
            'array<struct<name:string,dataset:string,passed_records:bigint,failed_records:bigint>>')
) t AS exp
ORDER BY timestamp DESC;
```

ATENCAO `expect_or_fail` é o mais severo: uma linha reprovada derruba a atualização. Use só para regras de integridade que não admitem exceção, e investigue a origem antes de relaxar para `expect_or_drop`.

PONTO DE VERIFICACAO **Fim da Sprint 4: Silver calculando**

- Pipeline `fininsight_silver` verde, com as 8 tabelas `sv_*` populadas;
- `sv_retornos_volatilidade` com retorno log, SMA, volatilidade, RSI e z-score (com backfill de 2 anos, sem nulos além do início da série);
- `sv_carteiras_scd2` com duas versões de PETR4 na carteira conservadora;
- Métricas de expectations visíveis na aba Data quality.

Parte VI - Sprint 5: Camada Gold

30. Catálogo de tabelas e star schema

Catálogo completo de tabelas

Camada	Tabela	Conteúdo
Bronze	bz_b3_cotacoes	COTAHIST diário da B3
Bronze	bz_bcb_series	Selic, IPCA e dólar (BCB)
Bronze	bz_yfinance_precos	Preços diários (BR/US)
Bronze	bz_alphavantage_fx	USD/BRL (Alpha Vantage)
Bronze	bz_fred_indicadores	FEDFUNDS, CPI, 10Y (FRED)
Bronze	bz_coingecko_cripto	Cripto: preço, market cap, volume
Bronze	bz_noticias_raw	Notícias brutas (NewsAPI)
Bronze	bz_cotacoes_streaming	Cotações em tempo real (Event Hubs)
Silver	sv_precos_ativos	Preços unificados e deduplicados
Silver	sv_retornos_volatilidade	Retorno log, SMA, vol, RSI, z-score
Silver	sv_precos_com_macro	Preço + dólar + Selic
Silver	sv_indicadores_macro	Indicadores macro em formato longo
Silver	sv_cambio	Dólar diário
Silver	sv_cripto	Cripto padronizado
Silver	sv_noticias_processadas	Notícias limpas
Silver	sv_carteiras_scd2	Composição histórica de carteiras (SCD2)
Gold	dim_ativo, dim_data, dim_moeda	3 dimensões
Gold	fato_precos_diarios, fato_retornos, fato_volatilidade	Fatos de mercado
Gold	fato_risco_var_sharpe	VaR 95%, Sharpe, drawdown
Gold	fato_sentiment_noticias	Resumo e sentimento (AI Functions)
Gold	fato_score_ml, fato_insights_genai	Gravadas por notebooks (cap. 36, 37)

Star schema

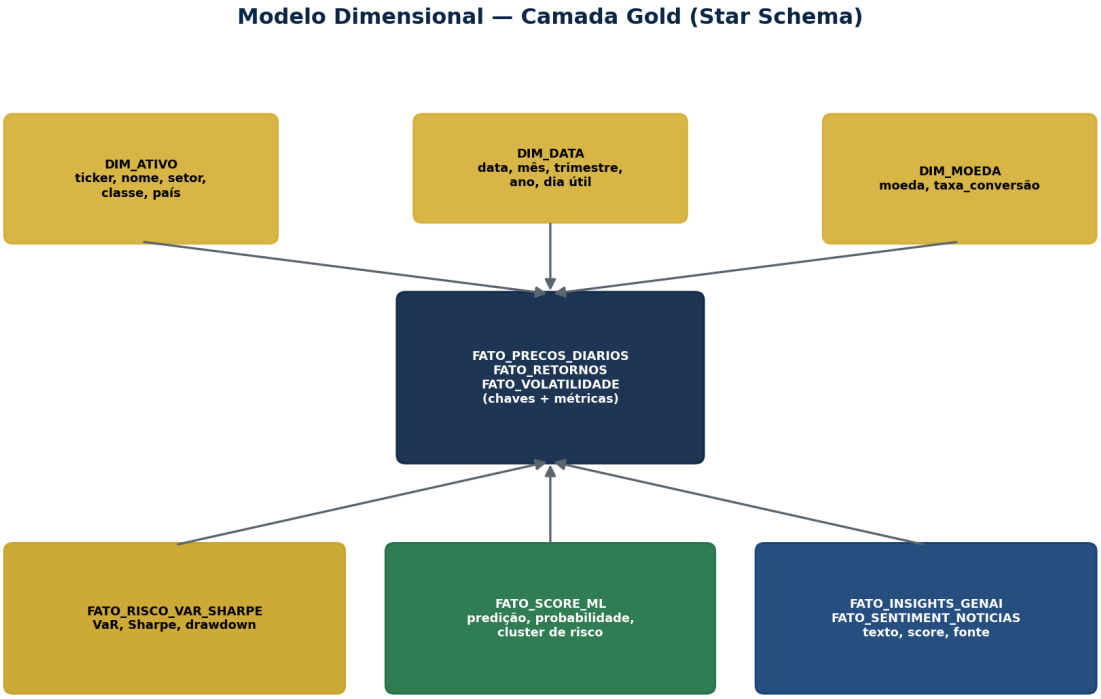


Figura 3 - Dimensões compartilhadas por todas as tabelas fato.

Chaves: `ativo_sk` é um **hash determinístico do ticker** (`xxhash64`), o mesmo valor em qualquer ambiente e sem depender de contador; `data_sk` é a própria data (tipo DATE).

31. Gold: dimensões

O pipeline `fininsight_gold` lê a Silver e grava no schema `gold`. As dimensões primeiro, pois os fatos fazem join com `dim_ativo`.

PASSO 1 dim_ativo

Junta os tickers vistos na Silver com um cadastro de referência (nome, setor, classe, país, moeda). Ticker sem cadastro entra como classe `desconhecida`.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/dim_ativo.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")

# Cadastro de referencia dos ativos (nome, setor, classe, pais, moeda)
CADASTRO = [
    ("PETR4.SA", "Petrobras PN", "Energia", "acao", "BR", "BRL"),
    ("VALE3.SA", "Vale ON", "Mineracao", "acao", "BR", "BRL"),
    ("ITUB4.SA", "Itau Unibanco PN", "Financeiro", "acao", "BR", "BRL"),
    ("^BVSP", "Ibovespa", "Indice", "indice", "BR", "BRL"),
    ("AAPL", "Apple", "Tecnologia", "acao", "US", "USD"),
    ("MSFT", "Microsoft", "Tecnologia", "acao", "US", "USD"),
]


@dp.materialized_view(
    name="dim_ativo",
    comment="Dimensao de ativos. ativo_sk = hash deterministico do ticker (igual em todo ambiente)",
    table_properties={"quality": "gold"},
)
def dim_ativo():
    cadastro = spark.createDataFrame(
        CADASTRO, "ticker string, nome string, setor string, classe string, pais string, moeda string"
    )
    vistos = spark.read.table(f"{CAT}.silver.sv_precos_ativos").select("ticker").distinct()
    return (
        vistos.join(cadastro, "ticker", "left")
        .withColumn("classe", F.coalesce("classe", F.lit("desconhecida")))
        .withColumn("ativo_sk", F.xxhash64("ticker"))
        .select("ativo_sk", "ticker", "nome", "setor", "classe", "pais", "moeda")
    )
```

PASSO 2 dim_data

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/dim_data.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F


@dp.materialized_view(
    name="dim_data",
    comment="Calendario 2020-2035 (dia util = segunda a sexta; feriados nao considerados)",
    table_properties={"quality": "gold"},
)
def dim_data():
    dias = spark.sql(
        "SELECT explode(sequence(to_date('2020-01-01'), to_date('2035-12-31'), "
        "interval 1 day)) AS data_sk"
    )
    return (
        dias.withColumn("ano", F.year("data_sk"))
        .withColumn("trimestre", F.quarter("data_sk"))
        .withColumn("mes", F.month("data_sk"))
        .withColumn("nome_mes", F.date_format("data_sk", "MMMM"))
        .withColumn("dia_semana", F.dayofweek("data_sk"))
        .withColumn("dia_util", F.dayofweek("data_sk").isin(2, 3, 4, 5, 6))
    )
```

NOTA `dia_util` considera segunda a sexta; feriados da B3 não estão no calendário. Para um calendário de pregões real, carregue a tabela de feriados da B3 e cruze aqui.

PASSO 3 dim_moeda

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/dim_moeda.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="dim_moeda",
    comment="Moedas e taxa de conversao para BRL (USD = ultimo dolar disponivel)",
    table_properties={"quality": "gold"},
)
def dim_moeda():
    ultimo_dolar = (
        spark.read.table(f"{CAT}.silver.sv_cambio")
        .orderBy(F.col("data_cotacao").desc())
        .limit(1)
        .select(F.col("ptax_venda").alias("taxa_para_brl"))
        .withColumn("moeda", F.lit("USD"))
        .withColumn("nome", F.lit("Dolar americano"))
    )
    brl = spark.createDataFrame(
        [("BRL", "Real", 1.0)], "moeda string, nome string, taxa_para_brl double"
    )
    return brl.unionByName(ultimo_dolar.select("moeda", "nome", "taxa_para_brl"))
```

32. Gold: fatos de preço, retorno e volatilidade

fato_precos_diarios

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_precos_diarios.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="fato_precos_diarios",
    comment="Fato: preco de fechamento e volume por ativo/dia",
    table_properties={"quality": "gold"},
)
@dp.expect_or_drop("preco_positivo", "preco_fechamento > 0")
def fato_precos_diarios():
    dim = spark.read.table("dim_ativo").select("ticker", "ativo_sk")
    return (
        spark.read.table(f"{CAT}.silver.sv_precos_ativos")
        .join(dim, "ticker")
        .select("ativo_sk", F.col("data_pregao").alias("data_sk"),
                "preco_fechamento", "volume", "origem")
    )
```

fato_retornos (inclui retorno acumulado)

A soma dos retornos logarítmicos é o retorno log acumulado; `exp(soma) - 1` converte em retorno simples acumulado.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_retornos.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="fato_retornos",
    comment="Fato: retorno logaritmico diario e retorno acumulado por ativo/dia",
    table_properties={"quality": "gold"},
)
def fato_retornos():
    dim = spark.read.table("dim_ativo").select("ticker", "ativo_sk")
    acumulada = (
        Window.partitionBy("ticker").orderBy("data_pregao")
        .rowsBetween(Window.unboundedPreceding, Window.currentRow)
    )
    return (
        spark.read.table(f"{CAT}.silver.sv_retornos_volatilidade")
        # soma de retornos log = retorno log acumulado; exp(x) - 1 converte para retorno simples
        .withColumn("retorno_acumulado", F.exp(F.sum("retorno_log_1d").over(acumulada)) - 1)
        .join(dim, "ticker")
        .select("ativo_sk", F.col("data_pregao").alias("data_sk"),
                "retorno_log_1d", "retorno_acumulado")
    )
```

fato_volatilidade

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_volatilidade.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")


@dp.materialized_view(
    name="fato_volatilidade",
    comment="Fato: medias moveis, volatilidade anualizada, RSI e z-score por ativo/dia",
    table_properties={"quality": "gold"},
)
def fato_volatilidade():
```

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_volatilidade.py

```
    dim = spark.read.table("dim_ativo").select("ticker", "ativo_sk")
    return (
        spark.read.table(f"{CAT}.silver.sv_retornos_volatilidade")
        .join(dim, "ticker")
        .select("ativo_sk", F.col("data_pregao").alias("data_sk"),
                "sma_21d", "sma_63d", "vol_21d_anualizada", "rsi_14d", "zscore_preco_63d")
    )
```

33. Gold: risco (VaR, Sharpe, drawdown), sentimento e o pipeline Gold

fato_risco_var_sharpe

KPI	Definição usada	Leitura
Retorno médio anual	média dos retornos log em 252 pregões x 252	Retorno esperado ao ano
Volatilidade anual	desvio-padrão dos retornos x raiz(252)	Risco total
Sharpe	(retorno anual - Selic anual) / volatilidade anual	Retorno por unidade de risco
VaR 95% (1 dia)	1,645 x volatilidade diária	Perda máxima esperada em 95% dos dias
Drawdown máximo	menor (preço - pico) / pico na janela	Pior queda desde um topo

> **NOTA** Janelas com menos de 20 observações são descartadas (`MIN_OBS`): estatística com poucos pontos engana. VaR paramétrico assume retornos normais; é uma aproximação didática.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_risco_var_sharpe.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")
MIN_OBS = 20      # minimo de retornos na janela para as metricas fazerem sentido
Z_95 = 1.645      # quantil da normal padrao para 95% de confianca


@dp.materialized_view(
    name="fato_risco_var_sharpe",
    comment="Fato: retorno/vol anuais, Sharpe, VaR 95% (1 dia) e drawdown maximo (252 pregoes)",
    table_properties={"quality": "gold"},
)
@dp.expect_or_fail("chave_fato_nao_nula", "ativo_sk IS NOT NULL AND data_sk IS NOT NULL")
@dp.expect("sharpe_calculado", "sharpe_ratio IS NOT NULL")      # so mede: nao derruba o pipeline
def fato_risco_var_sharpe():
    dim = spark.read.table("dim_ativo").select("ticker", "ativo_sk")
    j252 = Window.partitionBy("ticker").orderBy("data_pregao").rowsBetween(-251, 0)

    df = spark.read.table(f"{CAT}.silver.sv_precos_com_macro")
    df = df.withColumn("n_obs", F.count("retorno_log_1d").over(j252))
    df = df.withColumn("retorno_medio_anual", F.avg("retorno_log_1d").over(j252) * 252)
    df = df.withColumn("vol_anual", F.stddev("retorno_log_1d").over(j252) * F.lit(252 ** 0.5))

    # Sharpe = (retorno anual - taxa livre de risco anual) / volatilidade anual
    df = df.withColumn("rf_anual", F.col("rf_diaria_pct") / 100 * 252)
    df = df.withColumn(
        "sharpe_ratio", (F.col("retorno_medio_anual") - F.col("rf_anual")) / F.col("vol_anual")
    )

    # VaR parametrico 95% de 1 dia = z * volatilidade diaria (perda maxima esperada em 95% dos dias)
    df = df.withColumn("var_95_1d_pct", F.lit(Z_95) * F.col("vol_anual") / F.lit(252 ** 0.5))

    # drawdown: queda percentual desde o pico da janela; o maximo e o menor valor (mais negativo)
    df = df.withColumn("pico_movel", F.max("preco_fechamento").over(j252))
    df = df.withColumn(
        "drawdown_atual", (F.col("preco_fechamento") - F.col("pico_movel")) / F.col("pico_movel")
    )
    df = df.withColumn("drawdown_maximo_252d", F.min("drawdown_atual").over(j252))

    return (
        df.filter(F.col("n_obs") >= MIN_OBS)
        .join(dim, "ticker")
        .select("ativo_sk", F.col("data_pregao").alias("data_sk"), "retorno_medio_anual", "vol_anual",
                "sharpe_ratio", "var_95_1d_pct", "drawdown_maximo_252d")
    )
```

fato_sentiment_noticias

Usa as AI Functions do Databricks (`ai_summarize`, `ai_classify`, `ai_query`) diretamente no pipeline. O `sentiment_score` converte o rótulo em +1, 0, -1 para agregação.

PIPELINE DECLARATIVO (PYTHON) | VS Code, novo arquivo em pipelines/gold/ | pipelines/gold/fato_sentiment_noticias.py

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

CAT = spark.conf.get("fininsight.catalog", "fininsight_dev")
MODELO = "databricks-meta-llama-3-3-70b-instruct"   # endpoint de Foundation Model API do workspace

PERGUNTA = "De 0 a 100, qual o impacto desta noticia no preco do ativo? Responda so com o numero: "


@dp.materialized_view(
    name="fato_sentiment_noticias",
    comment="Fato: resumo, sentimento e impacto estimado por noticia/ativo (AI Functions)",
    table_properties={"quality": "gold"},
)
def fato_sentiment_noticias():
    dim = spark.read.table("dim_ativo").select("ticker", "ativo_sk")
    noticias = (
        spark.read.table(f"{CAT}.silver.sv_noticias_processadas")
        .filter("ticker_relacionado IS NOT NULL")
    )
    return (
        noticias.join(dim, noticias.ticker_relacionado == dim.ticker)
        .withColumn("resumo", F.expr("ai_summarize(texto_noticia, 40)"))
        .withColumn(
            "sentiment",
            F.expr("ai_classify(texto_noticia, array('positivo','neutro','negativo'))"),
        )
        .withColumn(
            "sentiment_score",
            F.when(F.col("sentiment") == "positivo", 1)
            .when(F.col("sentiment") == "negativo", -1)
            .otherwise(0),
        )
        .withColumn(
            "score_impacto",
            F.expr(f"try_cast(ai_query('{MODELO}', concat('{PERGUNTA}', texto_noticia)) AS INT)"),
        )
        .select("noticia_id", "ativo_sk", F.col("data_publicacao").alias("data_sk"), "titulo",
                "resumo", "sentiment", "sentiment_score", "score_impacto")
    )
```

CUSTO AI Functions cobram por token e rodam a cada atualização da tabela. Como a Silver de notícias é pequena (até 50 por coleta) o custo é baixo, mas não deixe isso em pipeline `Continuous` nem agendado de minuto em minuto.

NOTA O endpoint `databricks-meta-llama-3-3-70b-instruct` depende do que seu workspace/região oferece. Se der `endpoint not found`, veja **Serving** no menu e troque o nome na constante `MODELO`.

Criar o pipeline Gold e rodar ponta a ponta

1. Create, ETL pipeline: `fininsight_gold`; catálogo `fininsight_dev`, schema `gold`; source code: a pasta `pipelines/gold`; configuração `fininsight.catalog = fininsight_dev`.
2. Start. Ordem automática: `dim_ativo` primeiro, depois os fatos.
3. `fato_score_ml` e `fato_insights_genai` **não** são deste pipeline: são gravadas pelos notebooks 05 e 06 (cap. 36 e 37).

SQL | Databricks SQL Editor | verificação da Gold

```
SELECT a.ticker, a.setor, r.data_sk, round(r.sharpe_ratio, 2) AS sharpe,
       round(r.var_95_1d_pct * 100, 2) AS var95_pct, round(r.drawdown_maximo_252d * 100, 1) AS
           drawdown_pct
FROM fininsight_dev.gold.fato_risco_var_sharpe r
JOIN fininsight_dev.gold.dim_ativo a USING (ativo_sk)
ORDER BY r.data_sk DESC, a.ticker
LIMIT 20;
```

PONTO DE VERIFICACAO **Fim da Sprint 5: Gold com KPIs de risco**

- Pipeline `fininsight_gold` verde: 3 dimensões, `fato_precos_diarios`, `fato_retornos`, `fato_volatilidade`, `fato_risco_var_sharpe` e `fato_sentiment_noticias`;
- A consulta de verificação devolve Sharpe, VaR 95% e drawdown por ativo e data;
- `ativo_sk` consistente entre dimensão e fatos (join sem perda de linhas).

Parte VII - Sprint 6: AI/ML e GenAI

34. Feature Engineering

> **CODIGO DE REFERENCIA** Capítulos 34 a 37 são **código de referência**: dependem de histórico de dados (faça o backfill de 2 anos do cap. 15), de permissões no Unity Catalog e de recursos do seu workspace. Rode, leia o erro e ajuste.

As features de ML vêm das colunas que a Silver já calculou (retorno, volatilidade, RSI, z-score): **fonte única de verdade**, nada é recalculado no notebook de ML. A feature table fica no Unity Catalog, no schema `ml`.

PASSO 1 **Criar o schema ml e a feature table**

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo do Git folder e Run all | notebooks/04_feature_engineering.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 04 - Feature Engineering (Feature Table no Unity Catalog)
# MAGIC As features derivam da Silver (fonte unica de verdade): nenhuma formula e recalculada aqui.
# MAGIC Pre-requisito: pipeline `fininsight_silver` executado com historico (backfill `period="2y"`).

# COMMAND ----------

spark.sql("CREATE SCHEMA IF NOT EXISTS fininsight_dev.ml")

# COMMAND ----------

from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()
TABELA = "fininsight_dev.ml.features_ativos"

features = (
    spark.table("fininsight_dev.silver.sv_retornos_volatilidade")
    .select("ticker", "data_pregao", "retorno_log_1d", "sma_21d", "sma_63d",
            "vol_21d_anualizada", "rsi_14d", "zscore_preco_63d")
    .dropna(subset=["ticker", "data_pregao"])      # chaves primarias nao podem ser nulas
)

if not spark.catalog.tableExists(TABELA):
    fe.create_table(
        name=TABELA,
        primary_keys=["ticker", "data_pregao"],
        timestamp_keys=["data_pregao"],
        df=features,
        description="Features financeiras diarias por ativo (retorno, medias, vol, RSI, z-score)",
    )
else:
    fe.write_table(name=TABELA, df=features, mode="merge")

display(spark.table(TABELA).limit(10))
```

> **NOTA** `timestamp_keys=["data_pregao"]` marca a coluna temporal da feature table; as chaves primárias não podem ser nulas, por isso o `dropna`. Se o seu runtime pedir o pacote, instale `databricks-feature-engineering` com `%pip`.

35. Modelos preditivos com MLflow

Três modelos, cada um em um notebook da pasta `ml/`. O registro usa o **Unity Catalog** (`mlflow.set_registry_uri("databricks-uc")`): o modelo vira o objeto `fininsight_dev.ml.classificador_risco`, com versões e permissões como qualquer tabela.

Classificador de risco

Alvo: o ativo cai mais de 5% nos próximos 21 pregões. A separação treino/teste é **temporal** (treina no passado, testa no futuro), para não vazar informação do futuro para o treino.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all (precisa de histórico) | ml/classificador_risco.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Classificador de risco (Gradient Boosting)
# MAGIC Alvo: o ativo cai mais de 5% nos proximos 21 pregoes (`alto_risco = 1`).
# MAGIC Precisa de historico: rode a coleta com `PERIODO_YF = "2y"` e atualize a Silver antes.

# COMMAND ----------

import mlflow
import mlflow.sklearn
from mlflow.models import infer_signature
from pyspark.sql import functions as F
from pyspark.sql.window import Window
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, roc_auc_score

mlflow.set_registry_uri("databricks-uc")           # modelos versionados no Unity Catalog
MODELO = "fininsight_dev.ml.classificador_risco"
HORIZONTE = 21
LIMITE_QUEDA = -0.05
FEATURES = ["vol_21d_anualizada", "rsi_14d", "zscore_preco_63d", "retorno_log_1d"]

# COMMAND ----------

janela = Window.partitionBy("ticker").orderBy("data_pregao")
base = (
    spark.table("fininsight_dev.silver.sv_retornos_volatilidade")
    .withColumn("preco_futuro", F.lead("preco_fechamento", HORIZONTE).over(janela))
    .withColumn("retorno_futuro", F.col("preco_futuro") / F.col("preco_fechamento") - 1)
    .withColumn("alto_risco", (F.col("retorno_futuro") < LIMITE_QUEDA).cast("int"))
    .dropna(subset=FEATURES + ["retorno_futuro"])
)
pdf = (
    base.select("ticker", "data_pregao", *FEATURES, "alto_risco")
    .toPandas()
    .sort_values("data_pregao")
)
print(f"{len(pdf)} linhas | proporcao de alto risco: {pdf['alto_risco'].mean():.2%}")

# separacao TEMPORAL (treina no passado, testa no futuro) -- evita vazamento de informacao
corte = int(len(pdf) * 0.8)
treino, teste = pdf.iloc[:corte], pdf.iloc[corte:]

# COMMAND ----------

with mlflow.start_run(run_name="classificador_risco_gb"):
    clf = GradientBoostingClassifier(n_estimators=200, max_depth=3, learning_rate=0.05, random_state=42)
    clf.fit(treino[FEATURES], treino["alto_risco"])

    pred = clf.predict(teste[FEATURES])
    mlflow.log_params({"n_estimators": 200, "max_depth": 3, "learning_rate": 0.05,
                       "horizonte_pregoes": HORIZONTE, "limite_queda": LIMITE_QUEDA})
    mlflow.log_metric("accuracy", accuracy_score(teste["alto_risco"], pred))
    if teste["alto_risco"].nunique() > 1:                       # AUC exige as duas classes no teste
        proba = clf.predict_proba(teste[FEATURES])[:, 1]
        mlflow.log_metric("auc", roc_auc_score(teste["alto_risco"], proba))

    mlflow.sklearn.log_model(
        clf, artifact_path="modelo",
        signature=infer_signature(teste[FEATURES], pred),
        registered_model_name=MODELO,
    )
```

Previsão do dólar

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | ml/previsao_series_temporais.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Previsao do dolar (Prophet)
# MAGIC A serie `sv_cambio` vem do BCB com no maximo 20 pontos por coleta; para uma previsao util,
# MAGIC acumule varios dias de coleta ou amplie o historico (veja o capitulo 20).

# COMMAND ----------

# MAGIC %pip install prophet

# COMMAND ----------

dbutils.library.restartPython()

# COMMAND ----------

import mlflow
import mlflow.prophet
import pandas as pd
from prophet import Prophet
from pyspark.sql import functions as F

serie = (
    spark.table("fininsight_dev.silver.sv_cambio")
    .select(F.col("data_cotacao").alias("ds"), F.col("ptax_venda").alias("y"))
    .orderBy("ds")
    .toPandas()
)
serie["ds"] = pd.to_datetime(serie["ds"])

TESTE = 5                                   # ultimos 5 pontos ficam de fora para medir o erro
treino, teste = serie.iloc[:-TESTE], serie.iloc[-TESTE:]

# COMMAND ----------

with mlflow.start_run(run_name="previsao_dolar_prophet"):
    modelo = Prophet(daily_seasonality=False, weekly_seasonality=True)
    modelo.fit(treino)

    previsao = modelo.predict(teste[["ds"]])
    mae = (previsao["yhat"].values - teste["y"].values).__abs__().mean()
    mlflow.log_metric("mae", float(mae))
    mlflow.log_param("pontos_treino", len(treino))
    mlflow.prophet.log_model(modelo, artifact_path="modelo_dolar")

    futuro = modelo.make_future_dataframe(periods=10)
    display(modelo.predict(futuro)[["ds", "yhat", "yhat_lower", "yhat_upper"]].tail(10))
```

Clusterização de ativos

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | ml/clusterizacao_ativos.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Clusterizacao de ativos por perfil de risco/retorno (KMeans)

# COMMAND ----------

import mlflow
import mlflow.sklearn
from pyspark.sql import functions as F
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

perfil = (
    spark.table("fininsight_dev.silver.sv_retornos_volatilidade")
    .groupBy("ticker")
    .agg(F.avg("retorno_log_1d").alias("retorno_medio"), F.avg("vol_21d_anualizada").alias("vol_media"))
    .dropna()
    .toPandas()
    .set_index("ticker")
)

K = min(3, len(perfil))                     # nao pode haver mais clusters que ativos
X = StandardScaler().fit_transform(perfil[["retorno_medio", "vol_media"]])
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | ml/clusterizacao_ativos.py

```
with mlflow.start_run(run_name="kmeans_ativos"):
    km = KMeans(n_clusters=K, random_state=42, n_init=10).fit(X)
    mlflow.log_param("k", K)
    mlflow.log_metric("inertia", float(km.inertia_))
    mlflow.sklearn.log_model(km, artifact_path="kmeans")

perfil["cluster_risco"] = km.labels_
resultado = spark.createDataFrame(perfil.reset_index())
resultado.write.mode("overwrite").saveAsTable("fininsight_dev.ml.cluster_ativos")
display(resultado.orderBy("cluster_risco", "vol_media"))
```

Etapa MLflow	O que faz
Tracking	Registra parâmetros, métricas e artefatos de cada execução (aba Experiments)
Registry (Unity Catalog)	Versiona o modelo; **aliases** (`champion`) apontam a versão em uso
Serving	Expõe o modelo como endpoint REST para scoring em tempo real

36. Promoção de modelo e batch scoring

> **CORRIGIDO** A v1 usava `transition_model_version_stage(... stage="Production")`. No Unity Catalog **stages não existem mais**: promove-se com alias.

PASSO 1 Promover a versão para champion

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | ml/promover_modelo.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # Promover modelo: aponta o alias `champion` para a versao mais recente
# MAGIC No Unity Catalog nao existem mais "stages" (Staging/Production): usa-se **alias**.

# COMMAND ----------

import mlflow
from mlflow import MlflowClient

mlflow.set_registry_uri("databricks-uc")
MODELO = "fininsight_dev.ml.classificador_risco"

client = MlflowClient()
versoes = client.search_model_versions(f"name = '{MODELO}'")
ultima = max(int(v.version) for v in versoes)

client.set_registered_model_alias(MODELO, "champion", ultima)
print(f"{MODELO}@champion -> versao {ultima}")
```

PASSO 2 Batch scoring grava a fato_score_ml

Carrega `models:/fininsight_dev.ml.classificador_risco@champion` como UDF do Spark, pontua o último pregão de cada ativo e grava a tabela Gold `fato_score_ml`.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all (ou task do job diário) | notebooks/05_batch_scoring.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 05 - Batch scoring: grava `gold.fato_score_ml`
# MAGIC Usa o modelo `@champion` (ver ml/promover_modelo.py) sobre o ultimo pregao de cada ativo.

# COMMAND ----------

import mlflow
from pyspark.sql import functions as F
from pyspark.sql.window import Window

mlflow.set_registry_uri("databricks-uc")
MODELO_URI = "models:/fininsight_dev.ml.classificador_risco@champion"
FEATURES = ["vol_21d_anualizada", "rsi_14d", "zscore_preco_63d", "retorno_log_1d"]

predict = mlflow.pyfunc.spark_udf(spark, MODELO_URI, result_type="int")

# COMMAND ----------

ultimo = (
    spark.table("fininsight_dev.silver.sv_retornos_volatilidade")
    .dropna(subset=FEATURES)
    .withColumn(
        "rn",
        F.row_number().over(Window.partitionBy("ticker").orderBy(F.col("data_pregao").desc())),
    )
    .filter("rn = 1")
)
dim = spark.table("fininsight_dev.gold.dim_ativo").select("ticker", "ativo_sk")

score = (
    ultimo.join(dim, "ticker")
    .withColumn("score_risco", predict(*[F.col(c) for c in FEATURES]))
    .select("ativo_sk", F.col("data_pregao").alias("data_sk"), "score_risco",
            F.current_timestamp().alias("calculado_em"))
)
(score.write.mode("overwrite").option("overwriteSchema", "true")
      .saveAsTable("fininsight_dev.gold.fato_score_ml"))
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all (ou task do job diário) | notebooks/05_batch_scoring.py

```
display(spark.table("fininsight_dev.gold.fato_score_ml"))
```

> **NOTA** Erro clássico de schema no scoring acontece quando as colunas de treino diferem das de scoring. Por isso a lista `FEATURES` é a mesma nos dois notebooks e as features vêm da mesma origem.

37. GenAI: insights, RAG e AI Functions

Insights diários em linguagem natural

Todo dia, após a Gold atualizar, o notebook 06 lê os KPIs mais recentes, monta um prompt e pede ao modelo um resumo executivo em português. O prompt segue como **dado** (coluna do DataFrame) e não é concatenado em uma string SQL, o que evita quebra por aspas e injeção.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all (ou task do job diário) | notebooks/06_genai_insights.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # 06 - Insights diarios em linguagem natural (grava `gold.fato_insights_genai`)

# COMMAND ----------

from pyspark.sql import functions as F

MODELO = "databricks-meta-llama-3-3-70b-instruct"   # endpoint de Foundation Model API

kpis = spark.sql("""
    SELECT a.ticker, round(r.sharpe_ratio, 2) AS sharpe, round(r.var_95_1d_pct * 100, 2) AS var95_pct,
           round(r.drawdown_maximo_252d * 100, 1) AS drawdown_pct
    FROM fininsight_dev.gold.fato_risco_var_sharpe r
    JOIN fininsight_dev.gold.dim_ativo a ON a.ativo_sk = r.ativo_sk
    WHERE r.data_sk = (SELECT max(data_sk) FROM fininsight_dev.gold.fato_risco_var_sharpe)
""")
data_ref = spark.sql(
    "SELECT max(data_sk) AS d FROM fininsight_dev.gold.fato_risco_var_sharpe"
).first()["d"]

texto_kpis = kpis.toPandas().to_json(orient="records")
prompt = (
    "Voce e um analista de mercado. Em ate 5 linhas, em portugues, resuma riscos e oportunidades "
    "do dia com base nestes KPIs (JSON). Cite os tickers com maior drawdown e menor Sharpe. "
    f"KPIs: {texto_kpis}"
)

# COMMAND ----------

# o prompt vai como DADO (coluna), nunca concatenado dentro do texto SQL
(
    spark.createDataFrame([(data_ref, prompt)], "data_sk DATE, prompt STRING")
    .withColumn("insight", F.expr(f"ai_query('{MODELO}', prompt)"))
    .withColumn("modelo", F.lit(MODELO))
    .withColumn("gerado_em", F.current_timestamp())
    .drop("prompt")
    .write.mode("append").option("mergeSchema", "true")
    .saveAsTable("fininsight_dev.gold.fato_insights_genai")
)
ultimos = spark.table("fininsight_dev.gold.fato_insights_genai").orderBy(F.col("gerado_em").desc())
display(ultimos.limit(3))
```

RAG sobre notícias com Vector Search

As notícias viram embeddings em um índice de Vector Search; o LLM responde combinando a pergunta com os trechos recuperados. O índice Delta Sync exige uma tabela Delta com Change Data Feed; como a Silver é gerenciada pelo pipeline, o notebook copia as notícias para `gold.noticias_vs`.

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | genai/vector_search_setup.py

```
# Databricks notebook source
# MAGIC %md
# MAGIC # RAG sobre noticias: indice de Vector Search
# MAGIC O indice Delta Sync exige tabela Delta com Change Data Feed. `silver.sv_noticias_processadas`
# MAGIC e gerenciada pelo pipeline, entao copiamos para uma tabela Delta propria.

# COMMAND ----------

# MAGIC %pip install databricks-vectorsearch

# COMMAND ----------

dbutils.library.restartPython()
```

NOTEBOOK DATABRICKS | Databricks, abrir o arquivo e Run all | genai/vector_search_setup.py

```
# COMMAND ----------

from databricks.vector_search.client import VectorSearchClient

ORIGEM = "fininsight_dev.gold.noticias_vs"
ENDPOINT = "fininsight_vs_endpoint"
INDICE = "fininsight_dev.gold.idx_noticias"

(spark.table("fininsight_dev.silver.sv_noticias_processadas")
      .select("noticia_id", "titulo", "texto_noticia", "ticker_relacionado", "data_publicacao")
      .write.mode("overwrite").saveAsTable(ORIGEM))
spark.sql(f"ALTER TABLE {ORIGEM} SET TBLPROPERTIES (delta.enableChangeDataFeed = true)")

# COMMAND ----------

vsc = VectorSearchClient()
if ENDPOINT not in [e["name"] for e in vsc.list_endpoints().get("endpoints", [])]:
    vsc.create_endpoint_and_wait(name=ENDPOINT, endpoint_type="STANDARD")

vsc.create_delta_sync_index_and_wait(
    endpoint_name=ENDPOINT,
    index_name=INDICE,
    source_table_name=ORIGEM,
    pipeline_type="TRIGGERED",
    primary_key="noticia_id",
    embedding_source_column="texto_noticia",
    embedding_model_endpoint_name="databricks-gte-large-en",
)

# COMMAND ----------

indice = vsc.get_index(ENDPOINT, INDICE)
res = indice.similarity_search(query_text="queda da PETR4 hoje",
                               columns=["titulo", "texto_noticia", "data_publicacao"], num_results=5)
for linha in res["result"]["data_array"]:
    print(linha[0], "|", linha[2])
```

> **NOTA** O modelo de embedding `databricks-gte-large-en` é voltado a inglês. Para textos em português, verifique em **Serving** se há um modelo de embedding multilíngue no seu workspace e troque a constante.

AI Functions x Azure OpenAI x modelo próprio

Opção	Vantagem	Quando usar
AI Functions (`ai_query`, `ai_summarize`, `ai_classify`)	SQL nativo, sem infra, pay-per-token	Padrão do projeto
Azure OpenAI (módulo Terraform do cap. 41)	Modelos GPT, contrato corporativo	Quando há orçamento/exigência Azure; exige endpoint externo no Serving
Modelo open-source em Model Serving	Controle total, dado não sai do workspace	Restrição de dados

Dashboards

Databricks SQL e Power BI leem a Gold pelo Unity Catalog usando um SQL Warehouse serverless. Esta materialized view consolida os KPIs:

SQL | Databricks SQL Editor (depois de ter fato_score_ml e fato_sentiment_noticias) | sql/08_vw_dashboard_executivo.sql

```
-- SQL Editor (warehouse). View materializada consumida pelo dashboard / Power BI.
CREATE OR REPLACE MATERIALIZED VIEW fininsight_dev.gold.vw_dashboard_executivo AS
SELECT
  a.ticker, a.setor, r.data_sk,
  r.sharpe_ratio, r.var_95_1d_pct, r.drawdown_maximo_252d,
  s.score_risco, n.sentiment_medio
FROM fininsight_dev.gold.fato_risco_var_sharpe r
JOIN fininsight_dev.gold.dim_ativo a ON a.ativo_sk = r.ativo_sk
LEFT JOIN fininsight_dev.gold.fato_score_ml s
  ON s.ativo_sk = r.ativo_sk AND s.data_sk = r.data_sk
LEFT JOIN (
  SELECT ativo_sk, data_sk, avg(sentiment_score) AS sentiment_medio
  FROM fininsight_dev.gold.fato_sentiment_noticias
```

SQL | Databricks SQL Editor (depois de ter fato_score_ml e fato_sentiment_noticias) | sql/08_vw_dashboard_executivo.sql

```
  GROUP BY ativo_sk, data_sk
) n ON n.ativo_sk = r.ativo_sk AND n.data_sk = r.data_sk;
```

- Retorno diário e acumulado; volatilidade 21d/63d; RSI 14d;
- VaR 95% (1 dia), drawdown máximo e Sharpe por ativo;
- Score de risco (ML) e sentimento médio de notícias por ticker e dia.

PONTO DE VERIFICACAO **Fim da Sprint 6: AI/ML e GenAI**

- Feature table `fininsight_dev.ml.features_ativos` criada;
- Modelo `fininsight_dev.ml.classificador_risco` registrado com o alias `champion`;
- `gold.fato_score_ml` e `gold.fato_insights_genai` gravadas pelos notebooks 05 e 06;
- (opcional) índice de Vector Search respondendo a uma busca de teste.

Parte VIII - Operação, Governança e Fechamento

38. Orquestração: Jobs e Databricks Asset Bundle

Um **job** encadeia as etapas na ordem certa: coleta, os três pipelines, e em paralelo o scoring e os insights. O arquivo `databricks.yml` na raiz descreve pipelines **e** job, e o Databricks CLI publica tudo no ambiente escolhido.

YAML | VS Code, arquivo novo na RAIZ do repositório | databricks.yml

```
# databricks.yml - Databricks Asset Bundle (raiz do repositorio)
# Comandos:  databricks bundle validate -t dev | deploy -t dev | run fininsight_job_diario -t dev
bundle:
  name: fininsight_platform

variables:
  catalog:
    description: Catalogo Unity Catalog do ambiente
    default: fininsight_dev

targets:
  dev:
    mode: development
    default: true
    workspace:
      # URL do workspace DEV (output databricks_workspace_url do Terraform)
      host: https://adb-XXXXXXXXXXXXXXXX.X.azuredatabricks.net
    variables:
      catalog: fininsight_dev

  hml:
    workspace:
      host: https://adb-HML.azuredatabricks.net
    variables:
      catalog: fininsight_hml

  prod:
    mode: production
    workspace:
      host: https://adb-PROD.azuredatabricks.net
    variables:
      catalog: fininsight_prod

resources:
  pipelines:
    fininsight_bronze:
      name: fininsight_bronze_${bundle.target}
      catalog: ${var.catalog}
      schema: bronze
      serverless: true
      libraries:
        - glob:
            include: ./pipelines/bronze/**
      configuration:
        fininsight.catalog: ${var.catalog}

    fininsight_silver:
      name: fininsight_silver_${bundle.target}
      catalog: ${var.catalog}
      schema: silver
      serverless: true
      libraries:
        - glob:
            include: ./pipelines/silver/**
      configuration:
        fininsight.catalog: ${var.catalog}

    fininsight_gold:
      name: fininsight_gold_${bundle.target}
      catalog: ${var.catalog}
      schema: gold
      serverless: true
      libraries:
```

YAML | VS Code, arquivo novo na RAIZ do repositório | databricks.yml

```
        - glob:
            include: ./pipelines/gold/**
      configuration:
        fininsight.catalog: ${var.catalog}

  jobs:
    fininsight_job_diario:
      name: fininsight_job_diario_${bundle.target}
      tasks:
        - task_key: coleta_apis
          notebook_task:
            notebook_path: ./notebooks/01_coleta_apis.py

        - task_key: bronze
          depends_on: [{ task_key: coleta_apis }]
          pipeline_task:
            pipeline_id: ${resources.pipelines.fininsight_bronze.id}

        - task_key: silver
          depends_on: [{ task_key: bronze }]
          pipeline_task:
            pipeline_id: ${resources.pipelines.fininsight_silver.id}

        - task_key: gold
          depends_on: [{ task_key: silver }]
          pipeline_task:
            pipeline_id: ${resources.pipelines.fininsight_gold.id}

        - task_key: scoring_ml
          depends_on: [{ task_key: gold }]
          notebook_task:
            notebook_path: ./notebooks/05_batch_scoring.py

        - task_key: insights_genai
          depends_on: [{ task_key: gold }]
          notebook_task:
            notebook_path: ./notebooks/06_genai_insights.py

      schedule:
        quartz_cron_expression: "0 0 7 ? * MON-FRI"
        timezone_id: America/Sao_Paulo
        pause_status: PAUSED          # mude para UNPAUSED quando quiser ativar
```

CODIGO DE REFERENCIA Não validei este arquivo contra o seu workspace: rode `databricks bundle validate -t dev` e corrija conforme a mensagem. A sintaxe `libraries: - glob: include:` exige um Databricks CLI recente; em versões antigas use uma entrada `- file: path:` por arquivo.

PASSO 1 Validar, publicar e executar

TERMINAL | VS Code | raiz do repositório (onde está o databricks.yml)

```
databricks auth login --host https://adb-XXXXXXXXXXXXXXXX.X.azuredatabricks.net
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run fininsight_job_diario -t dev
```

PASSO 2 Acompanhar

Jobs & Pipelines, **Job runs**: cada task aparece com seu status e logs. O agendamento está `PAUSED`; troque para `UNPAUSED` quando quiser ativar.

CUSTO Um job diário que liga 3 pipelines serverless e chamadas de IA custa dinheiro todo dia. Mantenha pausado durante o estudo e rode sob demanda.

39. CI/CD DEV, HML e PROD

Fluxo: `develop` publica em DEV, `release/*` em HML e `main` em PROD (manual). Pull requests rodam testes e `bundle validate`. Autenticação por **service principal** com OAuth (variáveis `DATABRICKS_CLIENT_ID` e `DATABRICKS_CLIENT_SECRET`), nunca token pessoal no repositório.

Bitbucket Pipelines

YAML | VS Code, arquivo novo na RAIZ do repositório | bitbucket-pipelines.yml

```yaml
# bitbucket-pipelines.yml (raiz). Variaveis do repositorio: DATABRICKS_HOST_*, DATABRICKS_CLIENT_ID,
# DATABRICKS_CLIENT_SECRET (service principal). Nunca coloque tokens no arquivo.
image: python:3.12

definitions:
  steps:
    - step: &instalar-cli
        name: Instalar Databricks CLI
        script:
          - curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh

pipelines:
  branches:
    develop:
      - step:
          name: Deploy DEV
          deployment: dev
          script:
            - curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
            - databricks bundle validate -t dev
            - databricks bundle deploy -t dev
    "release/*":
      - step:
          name: Deploy HML
          deployment: staging
          script:
            - curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
            - databricks bundle deploy -t hml
    main:
      - step:
          name: Deploy PROD
          deployment: production
          trigger: manual
          script:
            - curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
            - databricks bundle deploy -t prod
  pull-requests:
    "**":
      - step:
          name: Validar bundle e testes
          script:
            - pip install pytest pandas
            - pytest tests -q
            - curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh | sh
            - databricks bundle validate -t dev
```

GitHub Actions (equivalente, se o repositório estiver no GitHub)

YAML | VS Code, arquivo novo em .github/workflows/ | .github/workflows/deploy.yml

```yaml
# Equivalente ao bitbucket-pipelines.yml para quem usa GitHub (este projeto usa Git folder do GitHub).
name: deploy-databricks
on:
  push:
    branches: [develop, main]
  pull_request:

jobs:
  testes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install pytest pandas && pytest tests -q
```

YAML | VS Code, arquivo novo em .github/workflows/ | .github/workflows/deploy.yml

```yaml
  deploy:
    if: github.event_name == 'push'
    needs: testes
    runs-on: ubuntu-latest
    env:
      DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
      DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
      DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t ${{ github.ref_name == 'main' && 'prod' || 'dev' }}
```

DICA Proteja a branch `main`: Pull Request obrigatório com aprovação e testes passando. É o gate que impede um pipeline quebrado de chegar à PROD.

CODIGO DE REFERENCIA Os arquivos de CI são um ponto de partida; ajuste as variáveis de host por ambiente (`DATABRICKS_HOST_*`) e crie o service principal no Account Console.

40. Governança, linhagem e mascaramento (LGPD)

O Unity Catalog entrega, sem código extra: **permissões** por catálogo/schema/tabela, **linhagem** automática (landing, bronze, silver, gold, modelo) visível na aba Lineage do Catalog Explorer, **mascaramento de coluna** e **filtro de linha**.

SQL | Databricks SQL Editor, logado como admin do catálogo | sql/07_governanca.sql

```
-- SQL Editor, logado como admin do catalogo. Executar uma vez (ou versionar este script).

-- 1) Permissoes minimas para um grupo de analistas ler a Gold
GRANT USE CATALOG ON CATALOG fininsight_dev TO `analistas`;
GRANT USE SCHEMA, SELECT ON SCHEMA fininsight_dev.gold TO `analistas`;

-- 2) Mascaramento de coluna (LGPD): tabela de demonstracao com documento do cliente
CREATE TABLE IF NOT EXISTS fininsight_dev.gold.dim_cliente_demo (
  cliente_id INT, nome STRING, documento STRING
);
INSERT INTO fininsight_dev.gold.dim_cliente_demo VALUES
  (1, 'Maria Souza', '123.456.789-09'), (2, 'Joao Lima', '987.654.321-00');

CREATE OR REPLACE FUNCTION fininsight_dev.gold.mascara_documento(doc STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('admins') THEN doc
            ELSE concat('***.***.***-', right(doc, 2)) END;

ALTER TABLE fininsight_dev.gold.dim_cliente_demo
  ALTER COLUMN documento SET MASK fininsight_dev.gold.mascara_documento;

-- 3) Filtro de linhas: analistas veem apenas ativos brasileiros
CREATE OR REPLACE FUNCTION fininsight_dev.gold.filtro_pais(pais STRING)
RETURNS BOOLEAN
RETURN is_account_group_member('admins') OR pais = 'BR';

ALTER TABLE fininsight_dev.gold.dim_ativo SET ROW FILTER fininsight_dev.gold.filtro_pais ON (pais);

-- 4) Conferir quem pode o que
SHOW GRANTS ON SCHEMA fininsight_dev.gold;
```

> **NOTA** `is_account_group_member('admins')` e o grupo `analistas` precisam existir na conta; crie-os no Account Console, User management, Groups. A tabela `dim_cliente_demo` é fictícia, só para demonstrar o mascaramento de um documento (LGPD).

41. Módulos opcionais: Key Vault e Azure OpenAI

Dois módulos que **não** são necessários para o fluxo principal. Ative apenas se quiser: remova o `#` do bloco correspondente em `infra/main.tf` e rode `terraform apply`.

Key Vault

O Key Vault guarda segredos da plataforma (nesta versão, apenas provisionado; as chaves de API ficam no secret scope do Databricks). O módulo `network` é um marcador para uma futura rede privada e não cria recursos.

TERRAFORM | VS Code, arquivo novo | infra/modules/keyvault/main.tf

```
data "azurerm_client_config" "current" {}

resource "azurerm_key_vault" "kv" {
  name                = "kv-fininsight-${var.environment}"
  location            = var.location
  resource_group_name = var.resource_group_name
  tenant_id           = data.azurerm_client_config.current.tenant_id
  sku_name            = "standard"
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/keyvault/variables.tf

```
variable "environment" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}
```

TERRAFORM | VS Code, arquivo novo | infra/modules/network/main.tf

```
# Reservado para VNet/Private Endpoints, caso a Parte II da apostila
# evolua para uma configuracao com rede privada. Nao ha recursos aqui
# na versao atual do projeto -- todos os recursos usam endpoint publico.
```

42. Troubleshooting: todos os erros reais e as correções

Todos os erros abaixo aconteceram de verdade durante a construção. Procure pela mensagem.

Terraform e Azure

Erro / sintoma	Causa	Correção
`\` no fim da linha no PowerShell	Continuação de linha é o acento grave (`)	Comando em uma linha, ou use `)
Pastas `modules/*` sem `main.tf`	Arquivos não criados	Criar os arquivos (ou usar o repositório pronto)
`Too many command line arguments` no plan	Argumentos no PowerShell com o exe fora do PATH	`$argsList = @( "C:\terraform\terraform.exe" ) + $argsList`
`provider hashicorp/databricks not found`	Módulo sem `required_providers`	`versions.tf` no módulo unity-catalog
`azurerm_cognitive_deployment`: bloco `sku`	Argumento errado	Usar bloco `scale`
`gpt-4o-mini` descontinuado	Modelo aposentado	`gpt-5-mini` versão `2025-08-07`
`Standard` SKU não suportado	Modelo exige deployment global	`scale { type = "GlobalStandard" }`
Databricks SKU `standard` deprecated	Unity Catalog exige premium	`sku = "premium"`
Variáveis `databricks_token`/`subscription_id`	Sobras de versões anteriores	Remover; provider autentica por `azure_workspace_resource_id`
Limite de metastore por região	Já existe um metastore	Usar `metastore_id` existente; sem `databricks_metastore`
Assignment exige admin	Sem permissão de conta	Remover `databricks_metastore_assignment`; admin associa no Account Console
Container para o catálogo ausente	Faltava `unitycatalog`	Adicionar o container no módulo storage
`No value for required variable metastore_id`	Faltava a variável na raiz	Declarar em `variables.tf` e no tfvars
`Error acquiring the state lock`	Apply anterior travou	`terraform force-unlock <LOCK_ID>`
`LinkedAuthorizationFailed`	Role no escopo errado	EventGrid/Storage Account Contributor no Resource Group

Databricks, coleta e Bronze

Erro / sintoma	Causa	Correção
`SCHEMA_NOT_FOUND fininsight_dev.landing`	Schema/volume não existiam	Terraform (cap. 9) ou `CREATE SCHEMA` + `CREATE EXTERNAL VOLUME`
Git folder: permissão negada	Credencial GitHub incorreta	Recriar a credencial com token classic escopo `repo`
`personal_access_token cannot be set for GitHub Linked credentials`	Credencial vinculada ao app	Excluir e criar nova com PAT
`No module named yfinance`	Biblioteca ausente no compute	`%pip install yfinance` + `dbutils.library.restartPython()`
`OSError: non-existent directory`	Pasta do dia não existia	`os.makedirs(pasta, exist_ok=True)`
BCB `HTTP 400`	`/ultimos/N` com N > 20	Usar `/ultimos/20`
`eventhub_bootstrap_servers` indefinido	Variável sem declaração	Ler de `dbutils.secrets.get` (cap. 23)

Erro / sintoma	Causa	Correção
`Volume bronze._schemas does not exist`	`schemaLocation` apontando para volume inexistente	Remover `schemaLocation` no pipeline
`input_file_name` não suportado	Função indisponível no Unity Catalog	`col("_metadata.file_path")`
`Illegal Parquet type INT64 (TIMESTAMP(NANOS,false))`	Pandas grava timestamp em ns	`coerce_timestamps="ms"` e apagar o parquet antigo
`Secret does not exist ... eventhub-bootstrap`	Módulo eventhubs não aplicado	`terraform apply` do módulo (cap. 23)
`No module named 'pyspark.pipelines'`	Runtime/pipeline antigo sem a API nova	Usar pipeline serverless atual, ou trocar `dp` por `import dlt` (tabela do cap. 16)
Terraform `eventhubs`: scope `fininsight` não existe	Notebook 00 ainda não rodou	Rodar o notebook 00 (cria o scope) e repetir o apply
Pipeline roda arquivo de exemplo	`my_transformation.py` criado pelo assistente	Remover do source code
Pipeline falha em pasta vazia da landing	Fonte sem chave de API não gerou arquivos	Configurar a chave (cap. 14) ou tirar o arquivo `bz_*` do pipeline

Streaming Table, COPY INTO e ML

Erro / sintoma	Causa	Correção
COPY INTO não carrega nada na 2ª vez	Comportamento esperado (idempotente)	`force = true` só se quiser recarregar
`ALTER STREAMING TABLE ... SCHEDULE` em tabela de pipeline	Agendamento é do pipeline	Agendar o pipeline/job (cap. 21)
`UNRESOLVED_COLUMN` em Silver depois de nova coluna na Bronze	Schema evoluiu	Reiniciar atualização; revisar `select`
`expect_or_fail` interrompe o pipeline	Regra crítica violada	Investigar origem antes de relaxar
`endpoint not found` em `ai_query`	Modelo indisponível na região	Escolher outro em Serving e ajustar `MODELO`
`transition_model_version_stage` falha	Stages inexistentes no Unity Catalog	`set_registered_model_alias`
Scoring com erro de schema	Features de treino diferentes das de scoring	Mesma lista `FEATURES` e mesma fonte
`ai_query` com rate limit	Muitas chamadas simultâneas	Processar em lotes ou usar throughput provisionado

43. Checklist, custos e glossário

Checklist por sprint

Sprint / cap.	Você consegue...
Fundamentos (1-5)	Explicar Lakehouse, Delta e Medallion; montar a árvore do projeto
1 - Infra (6-12)	Provisionar storage, workspace e Unity Catalog com landing e conferir no Portal
2 - Bronze (13-17)	Clonar o repositório, configurar secrets, coletar 7 fontes e rodar o pipeline Bronze
3 - Ingestão (18-23)	Usar COPY INTO, Streaming Table com SCHEDULE e pipeline SQL, escolher a ferramenta certa e ler Event Hubs
4 - Silver (24-29)	Usar time travel/MERGE, calcular retorno/SMA/RSI/z-score, aplicar SCD2 e expectations
5 - Gold (30-33)	Montar o star schema e calcular VaR, Sharpe e drawdown
6 - AI/ML e GenAI (34-37)	Treinar, registrar com alias e pontuar um modelo; gerar insights e montar um índice de Vector Search
Operação (38-42)	Orquestrar com bundle, fazer deploy DEV, HML, PROD, aplicar governança e ativar os opcionais

Mapa de custos

Recurso	Quando cobra	Como controlar
Event Hubs Standard	Por hora, enquanto o namespace existir	`terraform destroy -target=module.eventhubs` quando não usar
Pipelines serverless / Streaming Table com SCHEDULE	Por uso, a cada atualização	Modo Development, `DROP SCHEDULE`, rodar sob demanda
Job diário	Cada execução liga pipelines e IA	Manter `PAUSED` durante o estudo
AI Functions e Azure OpenAI	Por token	Poucas linhas; evitar `Continuous`
Storage ADLS Gen2	Por GB armazenado	Baixo; `VACUUM` e limpar landing antiga
Vector Search endpoint	Por hora de endpoint	Excluir o endpoint ao terminar o estudo

Glossário

Termo	Significado
Medallion	Organização em camadas Bronze (bruto), Silver (padronizado/calculado) e Gold (modelo para consumo)
Auto Loader	Fonte `cloudFiles`: ingestão incremental de arquivos novos
Declarative Pipelines / DLT	Lakeflow Spark Declarative Pipelines: você declara tabelas e o Databricks orquestra
Streaming table	Tabela incremental alimentada por uma fonte de streaming
Materialized view	Resultado de uma consulta mantido e atualizado pelo Databricks
COPY INTO	Comando SQL idempotente de carga de arquivos
Unity Catalog	Governança centralizada: catálogos, schemas, permissões, linhagem
Volume	Pasta governada do Unity Catalog para arquivos
Access Connector	Identidade gerenciada que dá ao Databricks acesso ao storage
SCD Type 2	Dimensão que preserva o histórico de mudanças com período de validade
VaR	Value at Risk: perda máxima esperada em um nível de confiança
Sharpe	Retorno em excesso à taxa livre de risco por unidade de volatilidade
Drawdown	Queda percentual a partir do último pico
Alias (MLflow UC)	Rótulo móvel (`champion`) que aponta uma versão do modelo